OPENSHIFT 4.20

SENIOR ADMIN MASTERBOOK

AWS UPI | Operations | Security | Networking
Troubleshooting | Jenkins | Interview preparation

COMBINED EDITION • ORIGINAL MATERIAL PRESERVED

EXPANDED EDITION · ADDITIONAL CONTENT INDEX

Original 99-page book is preserved; new senior L3 chapters begin on page 101.

01 1. AWS UPI deployment: dependencies and execution

02 2. etcd, Raft consensus and disaster recovery

03 3. ClusterVersion Operator, upgrades and release governance

04 4. MachineConfig, MCO, MCP and RHCOS lifecycle

05 5. OVN-Kubernetes: pod networking and packet path

06 6. Services, Routes, TLS and DNS internals

07 7. RBAC, SCC, Secrets, OAuth and certificates

08 8. CSI, EBS, EFS and persistent storage

09 9. Operators, OLM and ClusterOperator diagnostics

10 10. Monitoring, alerts, logging and incident response

11 11. Jenkins to OpenShift: secure production CI/CD

12 12. GitOps, Argo CD and drift reconciliation

13 13. Capacity, performance and scheduler decisions

14 14. Disconnected installation and restricted-network operations

15 15. Operational security and compliance

16 16. Application deployment, YAML and lifecycle management

17 17. Incident simulations: L3 diagnostic decision tree

18 18. Senior interview practical assessment and model answers

19 Glossary · senior administration terms

TABLE OF CONTENTS

Core textbook: 1. Production architecture and deployment 6
Core textbook: 2. AWS foundation: identity, DNS and quotas 7
Core textbook: 3. VPC, subnets, routing and availability 8
Core textbook: 4. OpenShift installer, manifests and Ignition 9
Core textbook: 5. UPI CloudFormation stacks and bootstrap 11
Core textbook: 6. RHCOS, control plane and etcd internals 12
Core textbook: 7. Worker nodes, CSRs and installation 13
Core textbook: 8. OpenShift YAML: core resource model 14
Core textbook: 9. Deployments, rollouts, probes and 15
Core textbook: 10. Services, DNS, Routes and ingress 16
Core textbook: 11. Secrets, ConfigMaps and service 17
Core textbook: 12. OVN-Kubernetes networking and 18
Core textbook: 13. Storage: PV, PVC, CSI and image registry 19
Core textbook: 14. Identity, RBAC, SCC and cluster security 20
Core textbook: 15. Jenkins integration and secure CI/CD 21
Core textbook: 16. Cluster administration and day-two 23
Core textbook: 17. Monitoring, logs, events and 24
Core textbook: 18. Troubleshooting: installation and 25
Core textbook: 19. Backup, disaster recovery and upgrades 27
Core textbook: 20. Hands-on production labs and 28
Core textbook: Lab 7: RBAC least privilege 29
Core textbook: 21. Scenario-based interview questions and 30
Core textbook: 22. Glossary and reference links 32
AWS UPI deployment procedure appendix 34
Senior administrator expansion 43
Chapter 01 — Architecture and internals 44
Chapter 03 — Bootstrap, Ignition and RHCOS 46
Chapter 05 — Upgrades and change management 48
Chapter 07 — etcd quorum and recovery 50
Chapter 09 — Services, Routes, Ingress and TLS 52
Chapter 11 — Storage CSI, EBS and EFS 54
Chapter 13 — SCC and workload security 56
Chapter 15 — Monitoring, alerting and logs 58
Chapter 17 — Capacity, performance and autoscaling 60
Chapter 19 — Jenkins, CI/CD and GitOps 62
Chapter 21 — HA, backups, OADP and DR 64

TABLE OF CONTENTS

Deep-dive handbook (original) 70

01. AWS UPI release planning 72

03. VPC and DNS 74

05. Installer and Ignition 76

07. Workers and CSR 78

09. RHCOS and MachineConfig 80

11. Pods, Deployments and YAML 82

13. OVN-Kubernetes networking 84

15. Storage and CSI 86

17. Updates and CVO 88

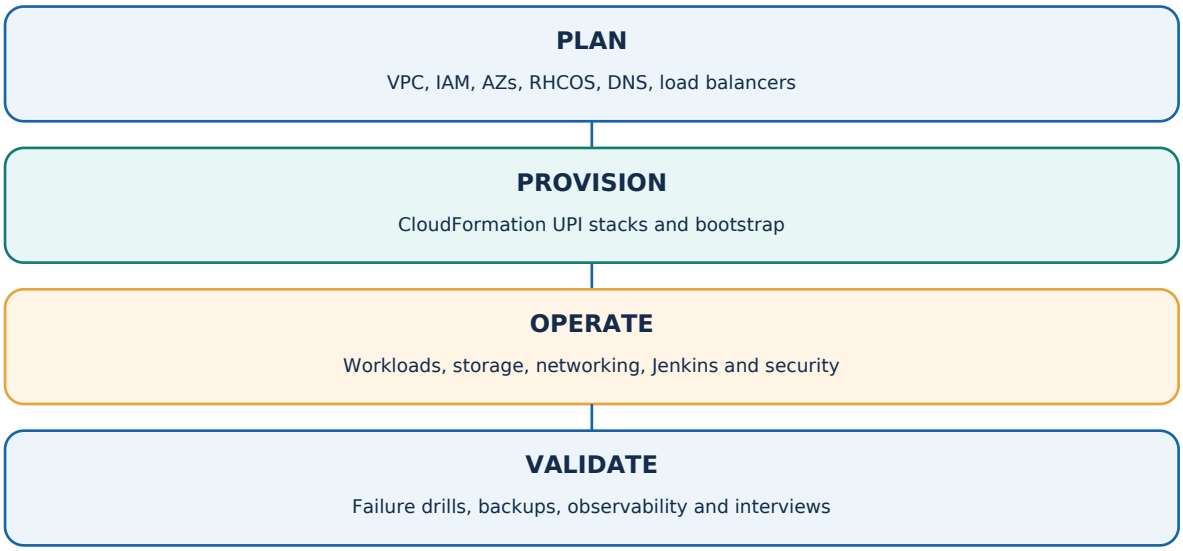
19. etcd and disaster recovery 90

21. Production troubleshooting 92

OPENSHIFT 4.20

AWS UPI PRODUCTION MASTERCLASS

Architecture • Step-by-step installation • YAML • Networking • Jenkins • Security • Administration • Troubleshooting • Job & interview readiness



Practical textbook | October 2026 | Illustrated reference edition

Important accuracy note. Terminal screenshots and diagrams in this book are illustrative reconstructions, not captures from a live AWS account. Placeholder values must be replaced. Official 4.20 UPI CloudFormation templates and parameter contracts must be checked against the exact chosen z-stream release before executing. AWS charges apply.

How to study: Read each principle, draw the traffic path, execute commands in a disposable lab, verify outputs, then perform the troubleshooting and interview drills. All commands require appropriate authorization.

Table of contents

01 1. Production architecture and deployment planning

02 2. AWS foundation: identity, DNS and quotas

03 3. VPC, subnets, routing and availability zones

04 4. OpenShift installer, manifests and Ignition

05 5. UPI CloudFormation stacks and bootstrap

06 6. RHCOS, control plane and etcd internals

07 7. Worker nodes, CSRs and installation validation

08 8. OpenShift YAML: core resource model

09 9. Deployments, rollouts, probes and autoscaling

10 10. Services, DNS, Routes and ingress

11 11. Secrets, ConfigMaps and service accounts

12 12. OVN-Kubernetes networking and NetworkPolicy

13 13. Storage: PV, PVC, CSI and image registry

14 14. Identity, RBAC, SCC and cluster security

15 15. Jenkins integration and secure CI/CD

16 16. Cluster administration and day-two operations

17 17. Monitoring, logs, events and observability

18 18. Troubleshooting: installation and runtime

19 19. Backup, disaster recovery and upgrades

20 20. Hands-on production labs and acceptance tests

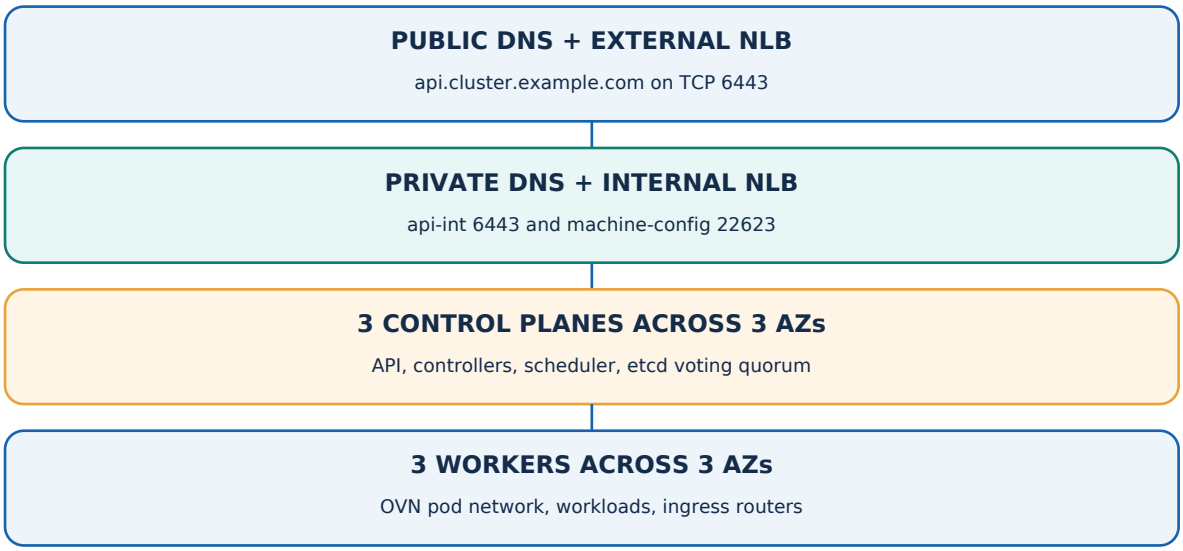
21 21. Scenario-based interview questions and answers

22 22. Glossary and reference links

The table of contents lists chapter order; every page has a printed page number. Use PDF search for topics and command names.

1. Production architecture and deployment planning

A production OpenShift cluster is a distributed system. Control-plane availability depends on etcd quorum, API access, node reachability, storage, DNS, and operator reconciliation. UPI means your team provisions AWS dependencies and owns their lifecycle.



Layer	Responsibility	Failure mode
AWS VPC	Private/public subnets, NAT, IGW, route tables	Node cannot reach endpoints
DNS + NLB	API, MCS, ingress traffic	API timeout or apps unavailable
RHCOS + Ignition	Immutable-oriented node boot configuration	Bootstrap or node join fails
OpenShift operators	Reconcile desired state	Degraded or progressing operators

Sizing is workload-dependent. Three masters plus three workers is a common HA starting point, not a guaranteed capacity target. Keep each master in a distinct availability zone.

Q. Why do three masters tolerate only one etcd failure?

Answer: etcd uses majority quorum: floor(3/2)+1 = 2. Losing two members leaves one, which cannot elect or commit safely.

2. AWS foundation: identity, DNS and quotas

Use short-lived credentials from IAM Identity Center where possible. The installation identity requires CloudFormation, IAM, EC2, ELB, Route 53, S3 and associated actions per the official installation permission model. Avoid embedding long-lived keys in files.

```
aws configure sso --profile ocp-prod
aws sso login --profile ocp-prod
export AWS_PROFILE=ocp-prod AWS_REGION=ap-south-1
aws sts get-caller-identity
aws ec2 describe-availability-zones --region ap-south-1
```

Verify quotas for vCPUs, EIPs, load balancers, VPCs and NAT gateways. Confirm domain delegation and public hosted zone. Route 53 internal hosted zone resolves api-int only within associated VPCs.

Control	Verification
Identity	aws sts get-caller-identity
Zone	aws route53 list-hosted-zones-by-name --dns-name example.com
Quota	AWS Service Quotas console/API
AMI	openshift-install coreos print-stream-json

Q. Why prefer IAM roles to instance access keys?

Answer: Instance roles provide rotating credentials through instance metadata, avoid hardcoded secrets and support scoped permissions.

3. VPC, subnets, routing and availability zones

The VPC hosts public load-balancing/NAT resources and private OpenShift machines. Private subnets have routes to per-AZ NAT gateways for internet egress, or a carefully designed private connectivity solution. DNS hostnames and DNS resolution must be enabled.

Subnet	Example CIDR	Placement
Public A/B/C	10.0.1.0/24, .2.0/24, .3.0/24	NLB/NAT
Private A/B/C	10.0.10.0/24, .20.0/24, .30.0/24	masters and workers
Pods	10.128.0.0/14	OVN overlay
Services	172.30.0.0/16	ClusterIP virtual addresses

```
aws ec2 describe-vpcs --vpc-ids "$VPC_ID"
aws ec2 describe-subnets --filters Name=vpc-id,Values="$VPC_ID"
aws ec2 describe-route-tables --filters Name=vpc-id,Values="$VPC_ID"
aws ec2 describe-nat-gateways --filter Name=vpc-id,Values="$VPC_ID"
```

Verify public default routes to IGW and private default routes to the appropriate NAT. Verify that VPC, pod, service and corporate CIDRs do not overlap. For strict private clusters, plan VPC endpoints and registry mirroring; do not assume NAT is optional without validating all dependencies.

Q. What happens if NAT fails in one AZ?

Answer: Nodes in that AZ can lose outbound access to external registries/APIs; cross-AZ NAT routing may restore access but changes failure isolation and cost.

4. OpenShift installer, manifests and Ignition

The installer generates install-config, manifests, Ignition and cluster authentication assets. UPI workers are externally provisioned, so compute replicas are set to zero in the installer config.

```
mkdir -p ~/ocp-prod
openshift-install create install-config --dir ~/ocp-prod
cp ~/ocp-prod/install-config.yaml ~/install-config.backup.yaml
chmod 600 ~/install-config.backup.yaml
apiVersion: v1
baseDomain: example.com
metadata:
  name: ocp-prod
platform:
  aws:
    region: ap-south-1
controlPlane:
  name: master
  replicas: 3
  platform:
    aws:
      type: m6i.xlarge
compute:
- name: worker
  replicas: 0
  platform:
    aws:
      type: m6i.2xlarge
networking:
  networkType: OVNKubernetes
  machineNetwork:
  - cidr: 10.0.0.0/16
  clusterNetwork:
  - cidr: 10.128.0.0/14
    hostPrefix: 23
  serviceNetwork:
  - 172.30.0.0/16
pullSecret: "REPLACE_WITH_REAL_JSON_SECRET"
sshKey: "ssh-ed25519 REPLACE_WITH_PUBLIC_KEY"
```

The pull secret is confidential. Never commit it. The API schema and optional parameters can vary by z-stream; validate against the installer. Set mastersSchedulable=false in generated scheduler configuration for normal production isolation.

```
openshift-install create manifests --dir ~/ocp-prod
openshift-install create ignition-configs --dir ~/ocp-prod
jq -r .infraID ~/ocp-prod/metadata.json
ls -l ~/ocp-prod/{bootstrap,master,worker}.ign
```

Ignition applies files and systemd configuration on first boot. The bootstrap Ignition contains sensitive material and has time limits. Protect the installation directory.

Q. What is the difference between Ignition and MachineConfig?

Answer: Ignition handles first-boot configuration. Machine Config Operator later manages supported host configuration changes and coordinated node reboots.

5. UPI CloudFormation stacks and bootstrap

Do not improvise UPI template contracts. Obtain the exact official 4.20 templates: VPC, network/load balancers, security, bootstrap, control-plane and compute. Their parameters, outputs and dependencies must be followed as documented.

Order	Stack	Expected output
1	VPC	VpcId, public/private subnet IDs
2	Network	NLBs, target groups, hosted zone, Lambda
3	Security	IAM profiles, security groups
4	Bootstrap	Temporary bootstrap EC2
5	Control plane	Three masters
6	Compute	Three workers after bootstrap progression

```
aws cloudformation validate-template --template-body file://vpc.yaml
aws cloudformation get-template-summary --template-body file://vpc.yaml
aws cloudformation create-stack --stack-name ocp-prod-vpc \
  --template-body file://vpc.yaml --parameters file://vpc-params.json
aws cloudformation wait stack-create-complete --stack-name ocp-prod-vpc
aws cloudformation describe-stacks --stack-name ocp-prod-vpc \
  --query "Stacks[0].Outputs"
```

The command pattern repeats for network, security, bootstrap, masters and workers with the appropriate parameter files and IAM capabilities. Never reuse a made-up JSON parameter key: use get-template-summary to discover the exact names.

Bootstrap joins temporary API and MCS targets. Once bootstrap-complete succeeds, deregister temporary targets and delete bootstrap resources according to the release procedure.

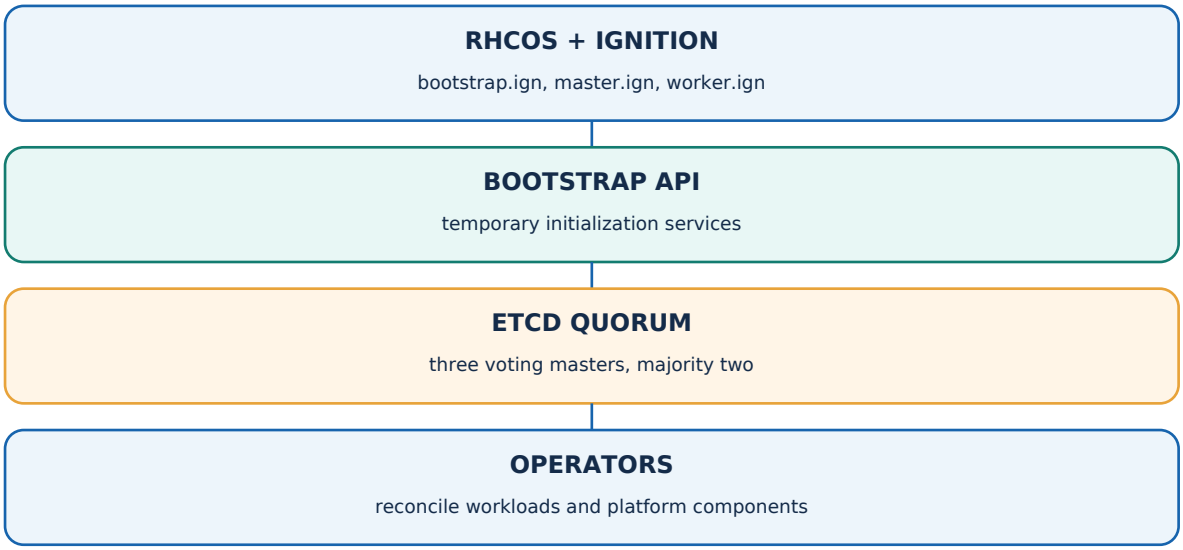
```
openshift-install wait-for bootstrap-complete --dir ~/ocp-prod --log-level=info
aws elbv2 describe-target-health --target-group-arn "$API_TG_ARN"
```

Q. Why must the bootstrap target be removed?

Answer: After control-plane handoff, the temporary bootstrap API/MCS endpoint is no longer part of the serving control plane; leaving stale targets can break health and routing.

6. RHCOS, control plane and etcd internals

RHCOS is the supported node OS for this workflow. The bootstrap machine starts temporary control-plane services; masters start etcd and the Kubernetes/OpenShift API components. etcd is a strongly consistent distributed key-value store that stores Kubernetes desired and observed state.




```
openshift-install coreos print-stream-json | jq .architectures.x86_64.images.aws
oc get pods -n openshift-etcd
oc get co etcd
oc get nodes -o wide
```

Never choose a random RHEL or RHCOS AMI. Select the matching image for the target region and architecture. Protect etcd backup keys and credentials; do not manually alter etcd data.

Q. What is reconciliation?

Answer: A controller repeatedly compares actual state with desired state and makes supported changes until they converge.

7. Worker nodes, CSRs and installation validation

Once masters are stable, workers boot with worker.ign and join through the Machine Config Server and API. Pending CSRs must be reviewed for expected node identities and certificate SANs before approval.

```
export KUBECONFIG=~/ocp-prod/auth/kubeconfig
oc get csr
oc describe csr CSR_NAME
oc adm certificate approve CSR_NAME
oc get nodes -o wide
oc get co
openshift-install wait-for install-complete --dir ~/ocp-prod
```

Do not approve every CSR automatically. Check node names, requested signer and expected source. The ingress DNS, ingress router scheduling and integrated registry storage must also be configured and verified before production use.

Check	Healthy indication
oc get nodes	All expected nodes Ready
oc get co	Available True, Progressing False, Degraded False
oc get clusterversion	Desired version available
oc get pods -A	No unexplained crash loops
oc get routes -A	Console and apps routes resolve

Q. Can Ready nodes coexist with a broken cluster?

Answer: Yes. Ready describes kubelet/node health, not necessarily ingress, storage, operator reconciliation or application correctness.

8. OpenShift YAML: core resource model

Kubernetes/OpenShift YAML is declarative: apiVersion chooses API group/version, kind selects object type, metadata names it, spec defines desired state, and status is controller-maintained observed state.

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: web
  namespace: demo
spec:
  replicas: 3
  selector:
    matchLabels:
      app: web
  template:
    metadata:
      labels:
        app: web
    spec:
      containers:
      - name: web
        image: registry.access.redhat.com/ubi9/httpd-24
        ports:
        - containerPort: 8080
```

Line-by-line: apiVersion=Deployment API; kind=controller type; metadata=name/namespace; replicas=desired pods; selector=which pods belong; template=pod blueprint; labels must match selector; containers=image runtime; containerPort=documentation/port metadata, not an external firewall rule.

```
oc apply -f deployment.yaml
oc get deploy web -n demo -o yaml
oc explain deployment.spec.strategy
oc diff -f deployment.yaml
oc rollout status deployment/web -n demo
```

Q. Why must selector labels match pod template labels?

Answer: The Deployment must identify the ReplicaSet/pods it manages; mismatch makes the resource invalid or unable to control intended pods.

9. Deployments, rollouts, probes and autoscaling

A Deployment manages stateless pods via ReplicaSets. RollingUpdate replaces pods incrementally; Recreate stops old pods before starting new ones. StatefulSet provides stable identity and storage associations; DaemonSet schedules a pod per eligible node.

```
oc create namespace demo
oc apply -f deployment.yaml
oc scale deployment web --replicas=5 -n demo
oc set image deployment/web web=registry.access.redhat.com/ubi9/httpd-24 -n demo
oc rollout history deployment/web -n demo
oc rollout undo deployment/web -n demo
readinessProbe:
  httpGet:
    path: /
    port: 8080
  initialDelaySeconds: 5
  periodSeconds: 10
livenessProbe:
  httpGet:
    path: /
    port: 8080
  initialDelaySeconds: 20
  periodSeconds: 10
resources:
  requests:
    cpu: 100m
    memory: 128Mi
  limits:
    memory: 512Mi
```

Readiness controls service endpoint inclusion. Liveness restarts stuck containers. Startup probe protects slow initialization. CPU request influences scheduling; CPU limit can throttle; memory limit can trigger OOMKilled. Tune based on measurements.

Q. Why is a readiness probe not the same as liveness?

Answer: Readiness removes a pod from traffic while it recovers; liveness causes a restart if the process is irrecoverably unhealthy.

10. Services, DNS, Routes and ingress

Service provides a stable virtual address and load-balances to selected pod endpoints. ClusterIP is internal; NodePort exposes node ports; LoadBalancer delegates to cloud integration; OpenShift Route publishes HTTP/HTTPS via the ingress controller.

```
apiVersion: v1
kind: Service
metadata:
  name: web
  namespace: demo
spec:
  selector:
    app: web
  ports:
  - name: http
    port: 80
    targetPort: 8080
  type: ClusterIP
apiVersion: route.openshift.io/v1
kind: Route
metadata:
  name: web
  namespace: demo
spec:
  to:
    kind: Service
    name: web
  port:
    targetPort: http
  tls:
    termination: edge
    insecureEdgeTerminationPolicy: Redirect
```

Service selector app=web must match pod labels. port=80 is service listener; targetPort=8080 is application listener. Edge TLS terminates at router; re-encrypt adds TLS to backend; passthrough forwards encrypted traffic to application.

```
oc apply -f service.yaml -f route.yaml
oc get svc,route,endpointslices -n demo
oc describe route web -n demo
oc get ingresscontroller -n openshift-ingress-operator
```

Q. Route vs Service?

Answer: Service exposes pods as a stable internal network endpoint. Route makes HTTP(S) service accessible through OpenShift ingress with host and TLS behavior.

11. Secrets, ConfigMaps and service accounts

ConfigMap stores non-sensitive configuration; Secret stores sensitive data as an API object. Kubernetes Secret base64 is encoding, not encryption. Enable encryption at rest, strict RBAC, external secret management where appropriate, and avoid logging values.

```
oc create secret generic app-db -n demo \
  --from-literal=username=appuser \
  --from-literal=password="REPLACE_WITH_SECURE_VALUE"
oc create configmap app-settings -n demo --from-literal=LOG_LEVEL=info
oc get secret app-db -n demo
oc describe secret app-db -n demo
apiVersion: v1
kind: Pod
metadata:
  name: example
  namespace: demo
spec:
  containers:
  - name: app
    image: registry.access.redhat.com/ubi9/httpd-24
    env:
    - name: DB_USER
      valueFrom:
        secretKeyRef:
          name: app-db
          key: username
```

Prefer mounted secret volumes for secrets that may rotate; environment variable changes generally require pod restart. Never place real passwords in shell history, manifest repositories or CI logs. The literal command above is a teaching illustration only.

Q. Is base64 secret data encrypted?

Answer: No. Base64 is reversible encoding; encryption at rest and API/RBAC protections are separate controls.

12. OVN-Kubernetes networking and NetworkPolicy

OVN-Kubernetes implements cluster pod networking, service traffic handling and policy enforcement. NetworkPolicy restricts selected pod ingress/egress; when a pod is isolated for a direction, only traffic allowed by applicable policies is permitted. DNS and required platform dependencies must be considered.

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: web-only
  namespace: demo
spec:
  podSelector:
    matchLabels:
      app: web
  policyTypes:
  - Ingress
  ingress:
  - from:
    - podSelector:
        matchLabels:
          role: frontend
    ports:
    - protocol: TCP
      port: 8080
```

This example permits inbound TCP 8080 from pods labeled role=frontend in the same namespace. It does not automatically permit ingress-router pods in another namespace; production policy must explicitly account for real source namespaces and paths.

```
oc get networkpolicy -A
oc get pods -n openshift-ovn-kubernetes
oc get svc -n openshift-dns
oc debug node/NODE_NAME
```

Q. Why can a service have healthy endpoints but time out?

Answer: NetworkPolicy, node networking, router-to-pod paths, wrong targetPort or application listening address may block traffic despite ready endpoints.

13. Storage: PV, PVC, CSI and image registry

PersistentVolume represents cluster storage, PVC is a claim, StorageClass selects provisioner and binding policy. AWS EBS CSI commonly supplies block volumes. EBS volumes are AZ-bound; understand scheduling and recovery implications. S3 is object storage, not a drop-in POSIX filesystem.

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: data
  namespace: demo
spec:
  accessModes:
  - ReadWriteOnce
  storageClassName: gp3-csi
  resources:
    requests:
      storage: 20Gi
```

Confirm the actual StorageClass name with oc get storageclass; gp3-csi is an example. WaitForFirstConsumer can defer provisioning until scheduling determines the AZ. For image registry, configure supported S3 object storage and verify registry operator status.

```
oc get storageclass
oc get pvc,pv -A
oc describe pvc data -n demo
oc get co storage
oc get configs.imageregistry.operator.openshift.io cluster -o yaml
```

Q. Why might PVC stay Pending?

Answer: Missing StorageClass, insufficient capacity/quota, CSI controller failures, topology conflict, or binding mode and scheduler constraints.

14. Identity, RBAC, SCC and cluster security

OpenShift authentication verifies identity; authorization determines permissions. RBAC Role/ClusterRole define allowed actions, and RoleBinding/ClusterRoleBinding grant them. SCC controls pod security constraints such as UID, capabilities, privilege and host access. Avoid granting privileged SCC broadly.

```
oc adm policy add-role-to-user view analyst -n demo
oc auth can-i get pods --as=analyst -n demo
oc get rolebindings -n demo
oc get scc
oc describe scc restricted-v2
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  name: pod-reader
  namespace: demo
rules:
- apiGroups: [""]
  resources: ["pods", "pods/log"]
  verbs: ["get", "list", "watch"]
```

A Role applies in a namespace. ClusterRole may describe cluster-scoped or reusable permissions. A service account is a workload identity, not a human login. Use scoped identities for CI and prefer short-lived projected tokens.

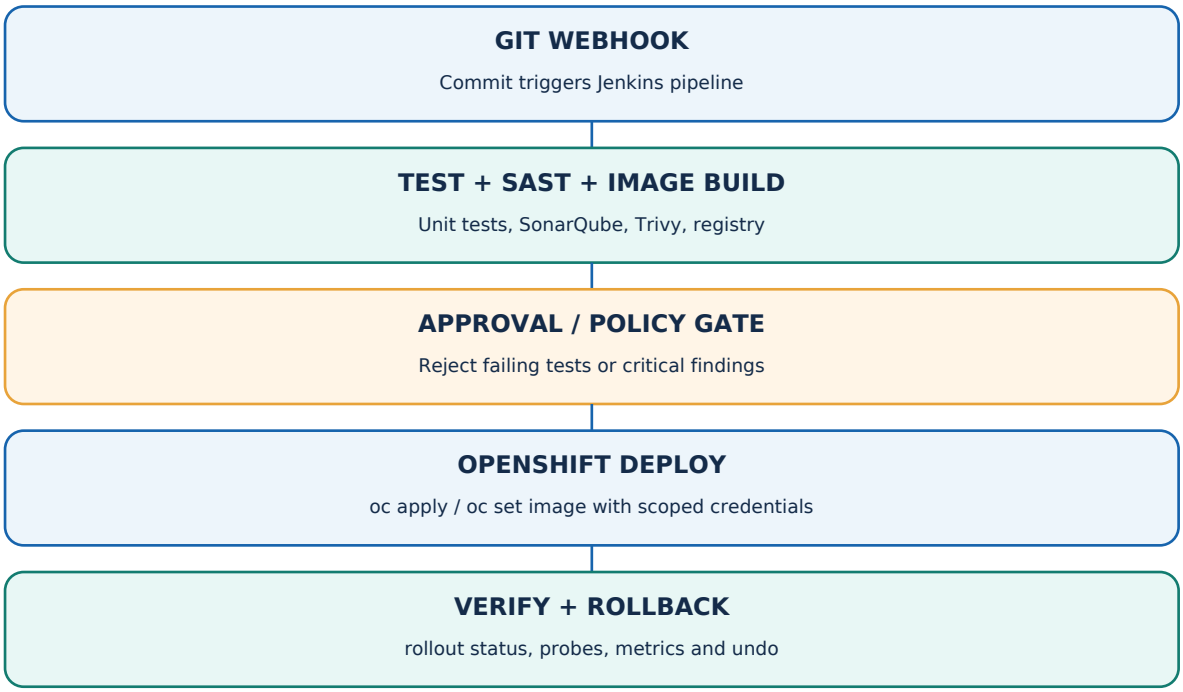
Production security checklist: restrict API access, enforce least privilege, protect etcd backups, configure audit logging, use TLS, scan images, pin trusted images by digest where appropriate, patch operators, control egress and monitor secrets access.

Q. RBAC vs SCC?

Answer: RBAC decides who can request API actions. SCC decides which security-sensitive pod configurations are admitted.

15. Jenkins integration and secure CI/CD

Jenkins is a CI orchestrator. A secure OpenShift pipeline checks out source, tests it, builds and scans an image, publishes it to a trusted registry, and deploys immutable image digests using a narrowly scoped OpenShift service account. Jenkins can run externally or on OpenShift; do not assume a built-in Jenkins operator is available or supported for your release.



Create a dedicated deployment namespace and service account; bind only permissions needed to update the deployment and read rollout status. Store CI credentials in Jenkins Credentials and prefer short-lived OpenShift tokens/OIDC when supported.

```
oc new-project cicd-demo
oc create sa jenkins-deployer -n cicd-demo
oc adm policy add-role-to-user edit \
  system:serviceaccount:cicd-demo:jenkins-deployer -n cicd-demo
oc auth can-i patch deployments --as=system:serviceaccount:cicd-demo:jenkins-deployer -n cicd-demo
```

The edit role is convenient for a lab but is too broad for hardened production. Replace it with a custom Role limited to the specific resources and verbs.

```
pipeline {
  agent any
  stages {
    stage("Test") { steps { sh "./scripts/test.sh" } }
    stage("Scan") { steps { sh "./scripts/scan.sh" } }
    stage("Build and Push") {
      steps { sh "./scripts/build-push.sh" }
    }
    stage("Deploy") {
      steps {
        withCredentials([string(credentialsId: "ocp-token", variable: "OCP_TOKEN")]) {
```

```
          sh """#!/bin/sh
            set +x
            oc login --token="$OCP_TOKEN" --server="$OCP_API"
            oc -n cicd-demo set image deployment/web web="$IMAGE_DIGEST"
            oc -n cicd-demo rollout status deployment/web --timeout=5m
          """
        }
      }
    }
  }
}
```

This is a teaching skeleton: test/scan/build scripts, OCP_API and IMAGE_DIGEST must be defined, TLS trust configured and credential exposure reviewed. For production, favor immutable digests, no privileged Docker socket, auditable promotion and separate approval for production.

Q. Why deploy by digest rather than mutable latest tag?

Answer: A digest identifies exact image content, enabling reproducible rollout, audit and rollback.

16. Cluster administration and day-two operations

Day-two administration means controlled maintenance, policy, capacity and recovery, not only checking pods. A typical shift includes operator health, node pressure, certificates, upgrades, backup verification, workload SLOs and audit review.

Operation	Commands
Cluster health	oc get co; oc get clusterversion
Nodes	oc get nodes; oc describe node NODE
Projects	oc get projects; oc new-project NAME
Capacity	oc adm top nodes; oc adm top pods -A
Cordoning	oc adm cordon NODE
Drain	oc adm drain NODE --ignore-daemonsets --delete-emptydir-data
Restore scheduling	oc adm uncordon NODE
Events	oc get events -A --sort-by=.lastTimestamp

Drain is disruptive. Respect PodDisruptionBudgets, local data, quorum, stateful workload failover and maintenance windows. On machine-managed clusters, reconcile with MachineConfig/Machine API workflows rather than editing hosts ad hoc.

Q. What is the difference between cordon and drain?

Answer: Cordon marks node unschedulable for new normal pods; drain attempts safe eviction of existing eligible pods.

17. Monitoring, logs, events and observability

Observability has metrics, logs, traces and events. Metrics reveal trends; logs show application and component messages; events record Kubernetes state transitions; traces reveal cross-service request latency. Use them together.

```
oc adm top nodes
oc adm top pods -A
oc get events -A --sort-by=.lastTimestamp
oc logs deployment/web -n demo --tail=100
oc logs pod/POD -c CONTAINER --previous -n demo
oc describe pod POD -n demo
oc get --raw /readyz
```

For production configure OpenShift monitoring, Alertmanager routing, log collection and retention, audit export and storage costs. Create alerts for etcd quorum, API latency, operator degradation, node filesystem pressure, PVC capacity and ingress error rates.

Q. Why are events insufficient for incident reconstruction?

Answer: Events are often short-lived and incomplete; correlate with persistent logs, metrics, deployment changes and cloud control-plane audit trails.

18. Troubleshooting: installation and runtime

Troubleshoot by following the dependency chain from client to DNS to NLB to router to Service to EndpointSlice to Pod to node to storage. Avoid restarting components without understanding the failure.

API timeout

Diagnostic commands: dig api...; aws elbv2 describe-target-health; nc -vz API 6443

Reasoning: DNS, NLB target health, security groups, API pods. Validate one hypothesis at a time, preserve logs and record the final root cause.

Q. What is the first step in a production incident?

Answer: Confirm impact, scope and timeline; gather read-only evidence before taking potentially disruptive action.

Bootstrap stuck

Diagnostic commands: openshift-install gather bootstrap --dir DIR

Reasoning: Ignition, MCS 22623, S3/IAM, bootstrap logs. Validate one hypothesis at a time, preserve logs and record the final root cause.

Q. What is the first step in a production incident?

Answer: Confirm impact, scope and timeline; gather read-only evidence before taking potentially disruptive action.

Pending pod

Diagnostic commands: oc describe pod POD; oc get events

Reasoning: Resource requests, taints, PVC, quotas. Validate one hypothesis at a time, preserve logs and record the final root cause.

Q. What is the first step in a production incident?

Answer: Confirm impact, scope and timeline; gather read-only evidence before taking potentially disruptive action.

ImagePullBackOff

Diagnostic commands: oc describe pod POD; oc get secret

Reasoning: Registry auth, DNS, proxy, image digest. Validate one hypothesis at a time, preserve logs and record the final root cause.

Q. What is the first step in a production incident?

Answer: Confirm impact, scope and timeline; gather read-only evidence before taking potentially disruptive action.

CrashLoopBackOff

Diagnostic commands: oc logs POD --previous; oc describe pod POD

Reasoning: Application exception, missing env, liveness. Validate one hypothesis at a time, preserve logs and record the final root cause.

Q. What is the first step in a production incident?

Answer: Confirm impact, scope and timeline; gather read-only evidence before taking potentially disruptive action.

OOMKilled

Diagnostic commands: oc describe pod POD; oc adm top pods

Reasoning: Memory limits, leak, spikes. Validate one hypothesis at a time, preserve logs and record the final root cause.

Q. What is the first step in a production incident?

Answer: Confirm impact, scope and timeline; gather read-only evidence before taking potentially disruptive action.

Route 503

Diagnostic commands: oc get endpointslices; oc get route; oc get svc

Reasoning: No ready backends, wrong targetPort, policy. Validate one hypothesis at a time, preserve logs and record the final root cause.

Q. What is the first step in a production incident?

Answer: Confirm impact, scope and timeline; gather read-only evidence before taking potentially disruptive action.

PVC Pending

Diagnostic commands: oc describe pvc PVC; oc get sc

Reasoning: CSI provisioner, AZ, storage class. Validate one hypothesis at a time, preserve logs and record the final root cause.

Q. What is the first step in a production incident?

Answer: Confirm impact, scope and timeline; gather read-only evidence before taking potentially disruptive action.

Operator Degraded

Diagnostic commands: oc describe co NAME; oc get pods -n NAMESPACE

Reasoning: Read conditions, dependencies, rollout. Validate one hypothesis at a time, preserve logs and record the final root cause.

Q. What is the first step in a production incident?

Answer: Confirm impact, scope and timeline; gather read-only evidence before taking potentially disruptive action.

19. Backup, disaster recovery and upgrades

An etcd backup is a consistent cluster-state snapshot plus required static resource data, not an application database backup. Application recovery also needs persistent data, image artifacts, secrets and external dependencies. Test restore in an isolated environment.

```
oc get clusterversion
oc get co
oc get nodes
oc adm must-gather --dest-dir=./must-gather
```

Perform etcd backup on an appropriate control-plane host using the Red Hat-supported 4.20 backup procedure and secure the generated snapshot and static resources. Never improvise restore steps during a live incident. Document recovery point objective (RPO), recovery time objective (RTO), AZ loss scenarios, and operator/cluster upgrade prerequisites.

Before upgrades, review supported upgrade paths, available updates, operator compatibility, capacity and PDBs; create validated backups; upgrade through supported channels and monitor operator progression. Rollback of OpenShift version is not generally a simple oc rollout undo operation.

Q. Why is an etcd snapshot not enough for full DR?

Answer: It does not capture all external volumes, object stores, application data or cloud dependencies; full recovery needs coordinated backup and infrastructure restoration.

20. Hands-on production labs and acceptance tests

Lab 1: Network audit

Enumerate subnets, routes, NAT and DNS; produce an AZ fault-domain diagram.

Acceptance evidence: Record starting state, command transcript, expected/actual results, failure symptoms, recovery actions and screenshot of final healthy resources.

Safety: Use a disposable namespace or test cluster; never inject failure into a production control plane.

Lab 2: Ignition lifecycle

Generate manifests and Ignition in a disposable installation directory; compare master vs worker role.

Acceptance evidence: Record starting state, command transcript, expected/actual results, failure symptoms, recovery actions and screenshot of final healthy resources.

Safety: Use a disposable namespace or test cluster; never inject failure into a production control plane.

Lab 3: Deploy three replicas

Create Deployment, Service and Route; test curl and rollback.

Acceptance evidence: Record starting state, command transcript, expected/actual results, failure symptoms, recovery actions and screenshot of final healthy resources.

Safety: Use a disposable namespace or test cluster; never inject failure into a production control plane.

Lab 4: Secret rotation

Mount a secret volume, rotate value, verify application reload semantics.

Acceptance evidence: Record starting state, command transcript, expected/actual results, failure symptoms, recovery actions and screenshot of final healthy resources.

Safety: Use a disposable namespace or test cluster; never inject failure into a production control plane.

Lab 5: Network isolation

Apply a default-deny policy in a lab namespace; selectively permit DNS and frontend.

Acceptance evidence: Record starting state, command transcript, expected/actual results, failure symptoms, recovery actions and screenshot of final healthy resources.

Safety: Use a disposable namespace or test cluster; never inject failure into a production control plane.

Lab 6: Storage topology

Provision PVC with WaitForFirstConsumer and inspect selected AZ.

Acceptance evidence: Record starting state, command transcript, expected/actual results, failure symptoms, recovery actions and screenshot of final healthy resources.

Safety: Use a disposable namespace or test cluster; never inject failure into a production control plane.

Lab 7: RBAC least privilege

Build custom Role and test oc auth can-i with impersonation.

Acceptance evidence: Record starting state, command transcript, expected/actual results, failure symptoms, recovery actions and screenshot of final healthy resources.

Safety: Use a disposable namespace or test cluster; never inject failure into a production control plane.

Lab 8: Jenkins pipeline

Run test, scan, build, push, deploy and rollout verification.

Acceptance evidence: Record starting state, command transcript, expected/actual results, failure symptoms, recovery actions and screenshot of final healthy resources.

Safety: Use a disposable namespace or test cluster; never inject failure into a production control plane.

Lab 9: Node drain

Cordon/drain a disposable worker with PDBs and validate service continuity.

Acceptance evidence: Record starting state, command transcript, expected/actual results, failure symptoms, recovery actions and screenshot of final healthy resources.

Safety: Use a disposable namespace or test cluster; never inject failure into a production control plane.

Lab 10: Failure injection

Break a test Service selector; diagnose no endpoints; restore configuration.

Acceptance evidence: Record starting state, command transcript, expected/actual results, failure symptoms, recovery actions and screenshot of final healthy resources.

Safety: Use a disposable namespace or test cluster; never inject failure into a production control plane.

21. Scenario-based interview questions and answers

Q. Explain OpenShift vs upstream Kubernetes.

Answer: OpenShift packages Kubernetes with integrated operators, security controls, routes, cluster lifecycle and opinionated operational capabilities.

Q. How does UPI differ from IPI?

Answer: UPI means you provision and manage underlying infrastructure; IPI installer provisions much of it.

Q. What happens when API NLB targets are unhealthy?

Answer: Clients cannot reach API even if instances exist; check health probes, target registration, security rules and API readiness.

Q. Why use three AZs?

Answer: To reduce correlated failures and spread etcd quorum and workloads across independent fault domains.

Q. How does Service select pods?

Answer: Its selector matches pod labels and EndpointSlice controllers track eligible backend endpoints.

Q. What is a Route?

Answer: An OpenShift resource exposing HTTP(S) service traffic through the ingress controller with host and TLS configuration.

Q. What is a Secret?

Answer: A Kubernetes object for sensitive values; base64 representation is not encryption.

Q. What is an SCC?

Answer: A security admission policy that restricts pod privileges and runtime security context.

Q. How do you investigate CrashLoopBackOff?

Answer: Describe pod, inspect previous logs, events, startup probes, configuration, resource limits and exit codes.

Q. What causes OOMKilled?

Answer: Container memory usage exceeded its enforced cgroup limit or memory pressure conditions caused termination.

Q. How does a Jenkins deployment authenticate?

Answer: With a dedicated scoped OpenShift identity, protected credentials and ideally short-lived tokens.

Q. How do you safely upgrade?

Answer: Check supported paths, backups, operator health, capacity, PDBs, dependencies and monitor update progression.

Q. What is etcd quorum?

Answer: A majority of voting members is required for consistent decisions and writes.

Q. What is a MachineConfig?

Answer: Declarative supported node OS configuration managed by Machine Config Operator.

Q. What is readiness vs liveness?

Answer: Readiness controls traffic eligibility; liveness restarts unhealthy containers.

Q. How do you debug a 503 Route?

Answer: Inspect router, route, service, EndpointSlice, pod readiness, TLS termination and network policies.

Q. Why are pods Pending?

Answer: Scheduling constraints, insufficient CPU/memory, taints, quota, PVC or topology conflicts.

Q. What is RBAC least privilege?

Answer: Grant only required verbs and resources within the smallest appropriate scope.

Q. What is a StatefulSet?

Answer: Controller providing stable pod identity, ordered behavior and persistent volume association.

Q. How do you verify production readiness?

Answer: Confirm nodes/operators, ingress, storage, security, backup restore, SLO monitoring and tested failover.

22. Glossary and reference links

API server — Kubernetes API endpoint for cluster resource requests.

Availability zone — Isolated AWS datacenter fault domain within a region.

Bootstrap — Temporary machine initializing the OpenShift control plane.

CloudFormation — AWS infrastructure-as-code stack service.

ClusterIP — Internal virtual IP of a Kubernetes Service.

ConfigMap — API object containing non-secret configuration.

CSI — Container Storage Interface for external storage integration.

CSR — Certificate Signing Request.

DaemonSet — Controller scheduling pods on eligible nodes.

Deployment — Controller for replicated stateless pods.

EndpointSlice — Scalable API representing Service backend endpoints.

etcd — Distributed consistent key-value store for Kubernetes state.

Ignition — RHCOS first-boot configuration mechanism.

Ingress controller — Router component handling application ingress traffic.

MachineConfig — Declarative RHCOS host configuration resource.

NAT gateway — AWS service providing outbound address translation.

NetworkPolicy — Pod-level ingress/egress traffic policy.

NLB — AWS Network Load Balancer for TCP traffic.

OVN-Kubernetes — OpenShift default software-defined networking implementation.

PersistentVolumeClaim — Request for storage capacity and access characteristics.

PodDisruptionBudget — Limit on voluntary disruptions to selected pods.

Pull secret — Credentials for retrieving container images.

RBAC — Role-Based Access Control.

RHCOS — Red Hat Enterprise Linux CoreOS.

Route — OpenShift HTTP(S) exposure resource.

SCC — Security Context Constraints.

ServiceAccount — Non-human Kubernetes workload identity.

StorageClass — Storage provisioning policy and provisioner selection.

UPI — User-Provisioned Infrastructure.

Official references and version checks

OpenShift 4.20 AWS UPI documentation
https://docs.redhat.com/en/documentation/openshift_container_platform/4.20/html/installing_on_aws/user-provisioned-infrastructure

OpenShift 4.20 documentation portal
https://docs.redhat.com/en/documentation/openshift_container_platform/4.20

Red Hat downloads
https://console.redhat.com/openshift/downloads

AWS CloudFormation documentation
https://docs.aws.amazon.com/AWSCloudFormation/latest/UserGuide/Welcome.html

Kubernetes API concepts
https://kubernetes.io/docs/concepts/

Jenkins Pipeline reference
https://www.jenkins.io/doc/book/pipeline/

Production caveat: template files, live AWS screenshots, real cloud outputs, and exact z-stream-specific parameter JSON were not retrieved from an AWS account. This textbook is an educational implementation guide, not evidence of a completed installation. Confirm all commands and manifests against official documentation before applying them.

AWS UPI DEPLOYMENT

OpenShift Container Platform 4.20 | Production Implementation Appendix

An expanded, command-led implementation companion to the 30-page masterclass. Includes validation, troubleshooting, rollback and operational handover. Hinglish explanations accompany technical instructions.

Important: AWS resources in this appendix are examples. Never run destructive commands in a shared account. Official Red Hat 4.20 CloudFormation templates and their exact parameters are the source of truth; this appendix does not pretend placeholder parameters are runnable.

Procedure contents

1. Readiness and security guardrails

2. Workstation and install assets

3. CloudFormation VPC

4. Network, load balancers and DNS

5. Security roles and profiles

6. RHCOS images and Ignition

7. Bootstrap and control-plane stacks

8. Worker stacks and CSR approvals

9. Ingress, image registry and completion

10. Verification, troubleshooting and recovery

11. Cleanup and production acceptance

12. Interview questions and answers

1. Readiness and security guardrails

UPI ka basic principle: OpenShift installer cluster manifests/Ignition banata hai, lekin AWS VPC, subnets, load balancers, DNS, security groups aur EC2 machines aap manage karte ho. Three masters in separate AZs give etcd one-member failure tolerance; three workers are a practical HA baseline, not a capacity guarantee.

Check	Production requirement
Identity	AWS account, approved IAM credentials, audit trail, AWS quota checks
DNS	Public Route 53 zone you control and internal DNS resolution in VPC
Connectivity	Required Red Hat registries and AWS endpoints; no unrestricted SSH or public 22623
Cost	NAT gateways, NLBs, EC2, EBS, data transfer, Red Hat subscriptions
Backup	Document etcd backup, S3 registry storage, cluster configuration backup
Versions	Same OpenShift 4.20 z-stream for installer/oc and matching RHCOS AMI

```
export AWS_REGION=ap-south-1
export AWS_PROFILE=ocp-prod
export CLUSTER_NAME=ocp-prod
export BASE_DOMAIN=example.com
export INSTALL_DIR="$HOME/ocp-prod"
mkdir -p "$INSTALL_DIR" "$HOME/ocp-cfn"
aws sts get-caller-identity
aws ec2 describe-availability-zones --region "$AWS_REGION"
```

Verify available zones in your account. AZ letters can map differently across accounts. Choose distinct AZ IDs and ensure each supports chosen EC2 family.

2. Workstation and installation assets

Download the OpenShift 4.20 installer and client from Red Hat console. Use a supported Linux workstation with AWS CLI v2, jq, tar, curl and SSH. Generate a dedicated key; only its public half goes into install-config.yaml.

```
aws --version
openshift-install version
oc version --client
ssh-keygen -t ed25519 -f "$HOME/.ssh/ocp-prod"
openshift-install create install-config --dir "$INSTALL_DIR"
```

Example install-config.yaml

```
apiVersion: v1
baseDomain: example.com
metadata:
  name: ocp-prod
platform:
  aws:
    region: ap-south-1
controlPlane:
  name: master
  replicas: 3
  platform:
    aws:
      type: m6i.xlarge
compute:
- name: worker
  replicas: 0
  platform: {}
networking:
  networkType: OVNKubernetes
```

```
  machineNetwork:
  - cidr: 10.0.0.0/16
  clusterNetwork:
  - cidr: 10.128.0.0/14
    hostPrefix: 23
  serviceNetwork:
  - 172.30.0.0/16
pullSecret: '<actual Red Hat JSON pull secret>'
sshKey: 'ssh-ed25519 <actual public key>'
```

compute.replicas=0 is intentional for manually created worker EC2s. Ensure platform and instance type fields match the specific Red Hat 4.20 UPI example. The YAML above is an explanatory baseline, not a guarantee of acceptance for every release configuration.

```
cp "$INSTALL_DIR/install-config.yaml" "$HOME/ocp-config-backup.yaml"
chmod 600 "$HOME/ocp-config-backup.yaml"
openshift-install create manifests --dir "$INSTALL_DIR"
grep -R mastersSchedulable "$INSTALL_DIR/manifests"
openshift-install create ignition-configs --dir "$INSTALL_DIR"
export INFRA_ID=$(jq -r .infraID "$INSTALL_DIR/metadata.json")
echo "$INFRA_ID"
```

For a 3-master + worker design, mastersSchedulable=false. Do not use the special three-node compact-cluster instructions, which require true and no workers. Generated Ignition contains short-lived certificates: begin deployment promptly, ideally within 12 hours.

3. CloudFormation VPC

Plan 10.0.0.0/16 VPC; public 10.0.1.0/24, 10.0.2.0/24, 10.0.3.0/24; private 10.0.10.0/24, 10.0.20.0/24, 10.0.30.0/24. For production use three NAT gateways, one per AZ, subject to cost and organizational policy.

From the official Red Hat documentation section 4.3, save the exact 4.20 **CloudFormation template for the VPC** as ~/ocp-cfn/vpc.yaml. Do not substitute a template from an older release. Template parameters and stack output names must be inspected, not guessed.

```
aws cloudformation validate-template \
  --template-body file://$HOME/ocp-cfn/vpc.yaml
aws cloudformation get-template-summary \
  --template-body file://$HOME/ocp-cfn/vpc.yaml \
  --query "Parameters[].{Key:ParameterKey,Default:DefaultValue}"
```

Create vpc-params.json using the actual keys returned above, with VPC CIDR, subnet CIDRs and chosen AZs. AWS CLI accepts a JSON array of ParameterKey/ParameterValue objects. Do not execute before replacing all placeholders.

```
aws cloudformation create-stack --stack-name ocp-prod-vpc \
  --template-body file://$HOME/ocp-cfn/vpc.yaml \
  --parameters file://$HOME/ocp-cfn/vpc-params.json
aws cloudformation wait stack-create-complete --stack-name ocp-prod-vpc
aws cloudformation describe-stacks --stack-name ocp-prod-vpc \
  --query "Stacks[0].Outputs"
```

Check subnet AZ placement, route table associations, internet gateway, NAT route, DNS hostnames and DNS resolution. Save the exact VPC and subnet IDs from stack outputs for subsequent stacks.

4. Network, load balancers and DNS

Load balancing has separate responsibilities: public API 6443 for authorized administrators, internal API 6443 and machine config server 22623 for nodes, and application ingress 80/443. Bootstrap initially participates in API/MCS target groups; it must be removed only after bootstrap completion. Do not expose MCS 22623 to the internet.

Save the official **network and load balancers** template as network.yaml and inspect required parameter keys. Supply actual VPC/subnet IDs, infra ID, public hosted zone and domain values. The template creates associated Route 53 and

ELB resources according to Red Hat release guidance.

```
aws cloudformation validate-template \
  --template-body file://$HOME/ocp-cfn/network.yaml
aws cloudformation get-template-summary \
  --template-body file://$HOME/ocp-cfn/network.yaml \
  --query "Parameters[].ParameterKey"
aws cloudformation create-stack --stack-name ocp-prod-network \
  --template-body file://$HOME/ocp-cfn/network.yaml \
  --parameters file://$HOME/ocp-cfn/network-params.json \
  --capabilities CAPABILITY_NAMED_IAM
aws cloudformation wait stack-create-complete --stack-name ocp-prod-network

aws cloudformation describe-stacks --stack-name ocp-prod-network \
  --query "Stacks[0].Outputs"
aws elbv2 describe-load-balancers --region "$AWS_REGION"
aws route53 list-hosted-zones-by-name --dns-name "$BASE_DOMAIN"
dig +short api.ocp-prod.example.com
```

DNS api-int is private and should be tested from inside VPC. API DNS can resolve before the API server starts; check target health and TCP connectivity separately.

5. Security roles and profiles

Save official security objects CloudFormation template as security.yaml. It defines required groups, instance profiles and roles. Confirm IAM permissions with your security team; a short-lived installation login alone does not guarantee the cluster has the credentials needed for ongoing AWS API use. Review the chosen supported Cloud Credential Operator mode carefully.

```
aws cloudformation get-template-summary \
  --template-body file://$HOME/ocp-cfn/security.yaml \
  --query "Parameters[].ParameterKey"
aws cloudformation create-stack --stack-name ocp-prod-security \
  --template-body file://$HOME/ocp-cfn/security.yaml \
  --parameters file://$HOME/ocp-cfn/security-params.json \
  --capabilities CAPABILITY_NAMED_IAM
aws cloudformation wait stack-create-complete --stack-name ocp-prod-security
aws cloudformation describe-stacks --stack-name ocp-prod-security \
  --query "Stacks[0].Outputs"
```

Review ingress/egress: SSH from bastion only, etcd 2379-2380 between masters, kubelet 10250, API 6443, MCS 22623 internal, OVN node communication, and router 80/443. Preserve template-required rules; a simplified port list is not sufficient.

6. RHCOS AMI and Ignition

```
openshift-install coreos print-stream-json > "$HOME/rhcos-stream.json"
jq ".architectures.x86_64.images.aws" "$HOME/rhcos-stream.json"
aws ec2 describe-images --image-ids ami-REPLACE \
  --query "Images[].{ID:ImageId,Arch:Architecture,State:State}"
```

AMI must match the selected OpenShift release and AWS region. bootstrap.ign, master.ign and worker.ign are distinct first-boot configuration files. Protect their contents and delete obsolete copies. For bootstrap S3 delivery, use private bucket + scoped read permission, not public-read ACLs.

```
export BOOTSTRAP_BUCKET="ocp-${AWS_ACCOUNT_ID}-bootstrap-unique"
aws s3api create-bucket --bucket "$BOOTSTRAP_BUCKET" \
  --region "$AWS_REGION" \
  --create-bucket-configuration LocationConstraint="$AWS_REGION"
aws s3api put-public-access-block --bucket "$BOOTSTRAP_BUCKET" \
  --public-access-block-configuration \

\BlockPublicAcls=true,IgnorePublicAcls=true,BlockPublicPolicy=true,RestrictPublicBuckets=tr
\ue
```

```
aws s3 cp "$INSTALL_DIR/bootstrap.ign" \
  "s3://$BOOTSTRAP_BUCKET/bootstrap.ign"
```

Enable bucket encryption and a least-privilege bootstrap instance role. The exact S3 Ignition delivery and instance user-data wiring must follow the official bootstrap template.

7. Bootstrap and control-plane machines

Save official bootstrap machine template as bootstrap.yaml. Populate its real parameters: infra ID, subnet, security group, RHCOS AMI, bootstrap ignition URL, load-balancer target groups and registration resources. Validate before stack creation.

```
aws cloudformation get-template-summary \
  --template-body file://$HOME/ocp-cfn/bootstrap.yaml \
  --query "Parameters[].ParameterKey"
aws cloudformation create-stack --stack-name ocp-prod-bootstrap \
  --template-body file://$HOME/ocp-cfn/bootstrap.yaml \
  --parameters file://$HOME/ocp-cfn/bootstrap-params.json \
  --capabilities CAPABILITY_NAMED_IAM
aws cloudformation wait stack-create-complete --stack-name ocp-prod-bootstrap
```

Save the official control-plane machine template as masters.yaml. Supply three distinct private subnets and release-matched AMI. The machines must receive master.ign or the supported pointer mechanism specified by the template.

```
aws cloudformation create-stack --stack-name ocp-prod-masters \
  --template-body file://$HOME/ocp-cfn/masters.yaml \
  --parameters file://$HOME/ocp-cfn/masters-params.json \
  --capabilities CAPABILITY_NAMED_IAM
aws cloudformation wait stack-create-complete --stack-name ocp-prod-masters
openshift-install wait-for bootstrap-complete \
  --dir "$INSTALL_DIR" --log-level=debug
```

Bootstrap complete ka meaning: temporary control plane handed control to permanent masters. It is NOT the same as full cluster readiness. Before deleting bootstrap, confirm target-group deregistration and the official 4.20 cleanup order.

8. Worker machines and certificate approvals

Save the official compute machines template as workers.yaml. Set three worker instances across three AZs and feed worker.ign. Some configurations require manual node certificate approvals. Never auto-approve all CSRs without checking identity.

```
aws cloudformation create-stack --stack-name ocp-prod-workers \
  --template-body file://$HOME/ocp-cfn/workers.yaml \
  --parameters file://$HOME/ocp-cfn/workers-params.json \
  --capabilities CAPABILITY_NAMED_IAM
aws cloudformation wait stack-create-complete --stack-name ocp-prod-workers
export KUBECONFIG="$INSTALL_DIR/auth/kubeconfig"
oc get csr
oc describe csr <CSR_NAME>
oc adm certificate approve <VERIFIED_CSR_NAME>
oc get nodes -o wide
```

CSR review: verify signerName, username, requested DNS/IP identities and node inventory. Client bootstrap and serving CSRs can be separate. Investigate unexpected requests rather than approving blindly.

9. Registry, ingress and installation completion

For UPI, configure the integrated image registry to use supported AWS S3 storage, including the right IAM access and bucket lifecycle controls. Set ingress wildcard DNS to the correct application ingress load balancer after the router

endpoint exists. Public API and app ingress are different endpoints.

```
oc get configs.imageregistry.operator.openshift.io cluster -o yaml
oc get co image-registry ingress
oc get ingresscontroller -n openshift-ingress-operator
oc get svc -n openshift-ingress
oc get routes -A
oc get clusterversion
oc get co
```

After bootstrap completion, delete only the temporary bootstrap stack and verify it is removed from load-balancer targets. Create required wildcard DNS records according to the official procedure. Wait for install-complete and record console URL and administrator credentials securely.

```
aws cloudformation delete-stack --stack-name ocp-prod-bootstrap
aws cloudformation wait stack-delete-complete \
  --stack-name ocp-prod-bootstrap
openshift-install wait-for install-complete \
  --dir "$INSTALL_DIR" --log-level=info
oc get nodes
oc get co
```

10. Troubleshooting decision tree

Symptom	Diagnosis and corrective direction
API DNS does not resolve	Check public/private Route 53 zones, record names, VPC resolver, association and DHCP options.
API 6443 timeout	Check NLB target health, listeners, subnet routing, SG/NACL, bootstrap/master kube-apiserver.
MCS 22623 unreachable	Check private NLB, internal DNS, security rules and bootstrap availability.
Master not joining	Inspect serial console, Ignition, kubelet journal, clock sync, RHCOS AMI and internal API.
etcd degraded	Inspect master connectivity, storage latency, quorum, etcd operator; never manually wipe data.
Workers NotReady	Inspect oc describe node, kubelet, CSR, OVN networking and registry pulls.
Ingress failing	Inspect router pods, route/service endpoints, wildcard DNS, 80/443 target health.
Registry degraded	Inspect S3 bucket, credentials, config operator and registry pod logs.

```
oc get nodes -o wide
oc get co
oc get pods -A --field-selector=status.phase!=Running
oc get events -A --sort-by=.lastTimestamp
oc describe co network
oc logs -n openshift-ingress deploy/router-default --tail=100
aws elbv2 describe-target-health --target-group-arn <REAL_TG_ARN>
```

Commands are diagnostic examples. The router deployment name and other resources can differ; list resources first. If bootstrap never completes, gather logs with openshift-install gather bootstrap and check AWS EC2 console output.

11. Cleanup and production acceptance

UPI cleanup is manual and must respect dependencies. Export cluster evidence and back up application data, PVs, S3 registry content and etcd using supported methods before any destructive operation. Delete compute stack, master stack, network/security stacks and VPC only when no shared resources remain.

```
aws cloudformation describe-stacks \
  --query "Stacks[].{Name:StackName,Status:StackStatus}"
aws cloudformation delete-stack --stack-name ocp-prod-workers
aws cloudformation wait stack-delete-complete --stack-name ocp-prod-workers
# Only after documented checks: masters, security/network, VPC
```

Production acceptance: three masters Ready in separate AZs; three workers Ready; all critical operators Available; image registry healthy; wildcard ingress works; RBAC least privilege; audit/log forwarding configured; tested etcd backup/restore runbook; alerting and patch/update process documented.

12. Interview questions and answers

Q: Why use UPI rather than IPI?

Answer: UPI gives control of VPC, DNS, load balancers, instance profiles and EC2 lifecycle. Tradeoff: infrastructure provisioning and cleanup are your responsibility.

Q: What does the bootstrap node do?

Answer: Runs a temporary control plane and serves initial configuration until permanent masters and etcd take over.

Q: Why three control-plane nodes?

Answer: Three etcd voters need two for quorum and can tolerate one voting member failure.

Q: Why are worker replicas zero in install-config?

Answer: Because worker EC2s are manually provisioned in this UPI procedure; three worker nodes are still created separately.

Q: Why does Ignition expire?

Answer: Its embedded bootstrap certificates have limited validity; start deployment promptly and regenerate assets when stale.

Q: Why are there public and internal API names?

Answer: Administrators can use the external API; nodes use private endpoints and MCS must stay internal.

Q: What is the difference between Route and Service?

Answer: Service provides stable cluster access to pods; Route publishes HTTP(S) applications through OpenShift ingress.

Q: Why use S3 for the registry?

Answer: Durable object storage supports registry blobs without relying on ephemeral pod filesystems.

Q: How do you investigate Pending CSRs?

Answer: Check signer, requester, node identity, expected IP/DNS SANs, then approve only legitimate requests.

Q: What is the safest bootstrap cleanup rule?

Answer: Wait for bootstrap-complete, verify LB registration transition, then remove temporary resources.

Q: How do you diagnose a NotReady node?

Answer: Check node conditions, kubelet/CRI-O logs, CNI/OVN, API reachability, certs and image pulls.

Q: How do you verify a production install?

Answer: Check nodes, cluster operators, clusterversion, ingress, registry, storage, monitoring and backup restore readiness.

Sources and implementation note

Official Red Hat 4.20 AWS UPI installation guide: https://docs.redhat.com/en/documentation/openshift_container_platform/4.20/html/installing_on_aws/user-provisioned-infrastructure

Official Red Hat 4.20 architecture/bootstrap: https://docs.redhat.com/en/documentation/openshift_container_platform/4.20/html/architecture/architecture-installation

Implementation boundary: CloudFormation template bodies, parameter JSON and AWS account-specific resource IDs are not bundled. The appendix documents how to obtain and validate them from the vendor. This prevents a false claim that the deployment is one-click executable. The actual install must be rehearsed and reviewed against the exact z-stream.

OPENSHIFT 4.20 | SENIOR ADMINISTRATOR EXPANSION

AWS UPI • Day-2 Operations • Security • Troubleshooting • Jenkins • Interview Preparation

This is an expanded companion volume appended to the previously created 39-page manual. It is a practical study reference, not a certified, fully tested production runbook. Release-sensitive operations must be checked against current Red Hat documentation before execution. All outputs and incident examples are illustrative.

CONTENTS

01. Architecture and internals

02. AWS UPI infrastructure and release verification

03. Bootstrap, Ignition and RHCOS

04. Cluster operators and CVO

05. Upgrades and change management

06. MachineConfig and MCP

07. etcd quorum and recovery

08. OVN-Kubernetes and pod networking

09. Services, Routes, Ingress and TLS

10. DNS and egress

11. Storage CSI, EBS and EFS

12. Authentication, OAuth and RBAC

13. SCC and workload security

14. Secrets, certificates and audit

15. Monitoring, alerting and logs

16. Node operations and scheduling

17. Capacity, performance and autoscaling

18. Application deployment, YAML and rollout

19. Jenkins, CI/CD and GitOps

20. Disconnected clusters and registry

21. HA, backups, OADP and DR

22. Incidents, RCA and senior operations

Appendix A. Deployment YAML and explanation; Appendix B. Jenkins pipeline; Appendix C. Glossary; Appendix D. Production readiness checklist

Chapter 01 — Architecture and internals

Concept and operating principle

Control plane runs kube-apiserver, kube-controller-manager, kube-scheduler, etcd, and OpenShift operators. Worker nodes run kubelet, CRI-O, OVN-Kubernetes and workloads. Cluster Version Operator (CVO) applies release payloads; Cluster Operators report health. Explain desired state: API objects are persisted in etcd; controllers reconcile actual state to desired state.

Step-by-step verification commands

```
oc get clusterversion -o yaml
oc get clusteroperators
oc get nodes -o wide
oc get pods -A --field-selector=status.phase!=Running
```

How to interpret the checks

Run these commands from an authenticated installation or administration workstation unless SSH/node debugging is specified. Start with read-only checks; compare observed conditions, events and timestamps against expected state. Confirm namespace and resource names before changes. Collect logs and evidence before remediation.

Production troubleshooting scenario

API is reachable but authentication fails: inspect authentication operator, OAuth pods, routes, DNS, certificates and ingress rather than restarting the API.

Response workflow

1. Establish impact and blast radius. 2. Inspect operator/node/resource conditions. 3. Correlate events and logs with recent changes. 4. Form and test a specific hypothesis. 5. Apply a supported corrective action. 6. Verify service recovery and document the RCA.

Senior interview Q&A

Q: How does a deployment reach desired replicas?

A: Deployment controller creates ReplicaSets, scheduler assigns pending pods, kubelet starts containers and readiness controls endpoints.

Follow-up interview challenge

Explain the expected command output, one false-positive interpretation, one security concern, and how you would validate the fix without introducing a second outage.

Chapter 02 — AWS UPI infrastructure and release verification

Concept and operating principle

UPI owns VPC, subnets, route tables, IAM, security groups, DNS, LBs, RHCOS machines and lifecycle. Use exact OpenShift 4.20 release-matched templates; this appendix does not substitute unverified parameter schemas. Validate IAM and quotas before creating stacks. Prefer short-lived AWS credentials and versioned infrastructure-as-code.

Step-by-step verification commands

```
aws sts get-caller-identity
openshift-install version
openshift-install coreos print-stream-json | jq .architectures.x86_64.images.aws
aws cloudformation validate-template --template-body file://vpc.yaml
aws cloudformation describe-stack-events --stack-name ocp-prod-vpc
```

How to interpret the checks

Run these commands from an authenticated installation or administration workstation unless SSH/node debugging is specified. Start with read-only checks; compare observed conditions, events and timestamps against expected state. Confirm namespace and resource names before changes. Collect logs and evidence before remediation.

Production troubleshooting scenario

Stack CREATE_FAILED: inspect first failure event, dependency, quota and permissions; avoid retrying with inconsistent partial resources.

Response workflow

1. Establish impact and blast radius. 2. Inspect operator/node/resource conditions. 3. Correlate events and logs with recent changes. 4. Form and test a specific hypothesis. 5. Apply a supported corrective action. 6. Verify service recovery and document the RCA.

Senior interview Q&A

Q: Why set compute replicas to zero in UPI?

A: Workers are created by the user-managed provisioning workflow; do not ask installer to provision them.

Follow-up interview challenge

Explain the expected command output, one false-positive interpretation, one security concern, and how you would validate the fix without introducing a second outage.

Chapter 03 — Bootstrap, Ignition and RHCOS

Concept and operating principle

Bootstrap temporarily hosts services needed to bring up control plane. Ignition applies first-boot configuration to RHCOS. Control-plane nodes must reach API and Machine Config Server; bootstrap can be removed only after installer confirms bootstrap-complete. RHCOS configuration is managed through MachineConfig rather than ad-hoc SSH changes.

Step-by-step verification commands

```
openshift-install create manifests --dir ocp-prod
openshift-install create ignition-configs --dir ocp-prod
openshift-install wait-for bootstrap-complete --dir ocp-prod --log-level=debug
ssh core@NODE sudo journalctl -b -u kubelet --no-pager
```

How to interpret the checks

Run these commands from an authenticated installation or administration workstation unless SSH/node debugging is specified. Start with read-only checks; compare observed conditions, events and timestamps against expected state. Confirm namespace and resource names before changes. Collect logs and evidence before remediation.

Production troubleshooting scenario

Bootstrap timeout: check API and MCS target health, internal DNS, security groups, Ignition access, time synchronization and kubelet logs.

Response workflow

1. Establish impact and blast radius. 2. Inspect operator/node/resource conditions. 3. Correlate events and logs with recent changes. 4. Form and test a specific hypothesis. 5. Apply a supported corrective action. 6. Verify service recovery and document the RCA.

Senior interview Q&A

Q: What is the difference between master.ign and worker.ign?

A: They contain role-specific initial configuration for control-plane and compute machines.

Follow-up interview challenge

Explain the expected command output, one false-positive interpretation, one security concern, and how you would validate the fix without introducing a second outage.

Chapter 04 — Cluster operators and CVO

Concept and operating principle

Cluster Operators expose Available, Progressing and Degraded conditions. CVO reconciles the release payload and cluster version. Inspect operator conditions and operand logs, not only pod status. An operator may be degraded while some pods are Running.

Step-by-step verification commands

```
oc get co
oc describe co ingress
oc get clusterversion -o yaml
oc get events -A --sort-by=.lastTimestamp
```

How to interpret the checks

Run these commands from an authenticated installation or administration workstation unless SSH/node debugging is specified. Start with read-only checks; compare observed conditions, events and timestamps against expected state. Confirm namespace and resource names before changes. Collect logs and evidence before remediation.

Production troubleshooting scenario

Ingress degraded: examine ingress controller status, router pod placement, certificates, DNS and LB health; correct the underlying dependency.

Response workflow

1. Establish impact and blast radius. 2. Inspect operator/node/resource conditions. 3. Correlate events and logs with recent changes. 4. Form and test a specific hypothesis. 5. Apply a supported corrective action. 6. Verify service recovery and document the RCA.

Senior interview Q&A

Q: How do CVO and OLM differ?

A: CVO manages platform release components; OLM manages supported lifecycle of add-on Operators.

Follow-up interview challenge

Explain the expected command output, one false-positive interpretation, one security concern, and how you would validate the fix without introducing a second outage.

Chapter 05 — Upgrades and change management

Concept and operating principle

Check supported upgrade path, channel, release recommendations, operator health, capacity, disruption budgets, backups and maintenance window. OpenShift upgrade is not an ordinary reversible deployment; never assume version rollback is supported. Track CVO and MCP progress and preserve incident evidence.

Step-by-step verification commands

```
oc adm upgrade
oc get clusterversion
oc get co
oc get mcp
oc get pdb -A
```

How to interpret the checks

Run these commands from an authenticated installation or administration workstation unless SSH/node debugging is specified. Start with read-only checks; compare observed conditions, events and timestamps against expected state. Confirm namespace and resource names before changes. Collect logs and evidence before remediation.

Production troubleshooting scenario

Upgrade stuck: identify degraded ClusterOperator or MachineConfigPool, check node drains and PDB constraints, resolve the blocker rather than forcing unsupported downgrade.

Response workflow

1. Establish impact and blast radius. 2. Inspect operator/node/resource conditions. 3. Correlate events and logs with recent changes. 4. Form and test a specific hypothesis. 5. Apply a supported corrective action. 6. Verify service recovery and document the RCA.

Senior interview Q&A

Q: How do you minimize application downtime during upgrade?

A: Multi-AZ replicas, readiness probes, PDBs, capacity headroom, supported upgrade path and monitored rolling node updates.

Follow-up interview challenge

Explain the expected command output, one false-positive interpretation, one security concern, and how you would validate the fix without introducing a second outage.

Chapter 06 — MachineConfig and MCP

Concept and operating principle

MachineConfig Operator renders OS configuration and rolls it out through MachineConfigPools. MCD runs on nodes. Changes can cause drains and reboots; test in nonproduction and consider dedicated pools. Inspect rendered config, node annotations and MCD logs for failures.

Step-by-step verification commands

```
oc get machineconfig
oc get mcp -o wide
oc describe mcp worker
oc get pods -n openshift-machine-config-operator
oc debug node/NODE -- chroot /host journalctl -u machine-config-daemon -b
```

How to interpret the checks

Run these commands from an authenticated installation or administration workstation unless SSH/node debugging is specified. Start with read-only checks; compare observed conditions, events and timestamps against expected state. Confirm namespace and resource names before changes. Collect logs and evidence before remediation.

Production troubleshooting scenario

MCP degraded: inspect degraded node, rendered config, MCD log, disk pressure, drain blockers and conflicting manual changes.

Response workflow

1. Establish impact and blast radius. 2. Inspect operator/node/resource conditions. 3. Correlate events and logs with recent changes. 4. Form and test a specific hypothesis. 5. Apply a supported corrective action. 6. Verify service recovery and document the RCA.

Senior interview Q&A

Q: What is a rendered MachineConfig?

A: Merged desired machine configuration generated from applicable MachineConfigs for a pool.

Follow-up interview challenge

Explain the expected command output, one false-positive interpretation, one security concern, and how you would validate the fix without introducing a second outage.

Chapter 07 — etcd quorum and recovery

Concept and operating principle

etcd is the consistent key-value store for API state. Three voting members require two for quorum. Losing two voting members usually makes writes unavailable. Take supported encrypted etcd snapshots with matching static-pod resources; securely store them off-cluster. Recovery is disruptive and must follow release-specific Red Hat instructions.

Step-by-step verification commands

```
oc get co etcd
oc get pods -n openshift-etcd -o wide
oc adm must-gather
oc get nodes -l node-role.kubernetes.io/master
```

How to interpret the checks

Run these commands from an authenticated installation or administration workstation unless SSH/node debugging is specified. Start with read-only checks; compare observed conditions, events and timestamps against expected state. Confirm namespace and resource names before changes. Collect logs and evidence before remediation.

Production troubleshooting scenario

Two masters fail: stabilize infrastructure, determine etcd quorum and membership, follow documented control-plane recovery rather than deleting members casually.

Response workflow

1. Establish impact and blast radius. 2. Inspect operator/node/resource conditions. 3. Correlate events and logs with recent changes. 4. Form and test a specific hypothesis. 5. Apply a supported corrective action. 6. Verify service recovery and document the RCA.

Senior interview Q&A

Q: Can a three-member etcd cluster survive two failures?

A: No; majority is two and only one remaining member cannot form quorum.

Follow-up interview challenge

Explain the expected command output, one false-positive interpretation, one security concern, and how you would validate the fix without introducing a second outage.

Chapter 08 — OVN-Kubernetes and pod networking

Concept and operating principle

Pod IPs come from clusterNetwork, while Services provide stable virtual IPs and endpoint selection. OVN programs logical switches, routers and policy. Pod-to-pod same/different node traffic may traverse overlays. Distinguish DNS lookup, TCP reachability, policy drops and application readiness.

Step-by-step verification commands

```
oc get pods -n openshift-ovn-kubernetes -o wide
oc get network.config cluster -o yaml
oc get networkpolicy -A
oc debug node/NODE
```

How to interpret the checks

Run these commands from an authenticated installation or administration workstation unless SSH/node debugging is specified. Start with read-only checks; compare observed conditions, events and timestamps against expected state. Confirm namespace and resource names before changes. Collect logs and evidence before remediation.

Production troubleshooting scenario

Cross-node pods cannot communicate: verify node reachability, OVN pods, encapsulation security-group rules, network policies, MTU and routes.

Response workflow

1. Establish impact and blast radius. 2. Inspect operator/node/resource conditions. 3. Correlate events and logs with recent changes. 4. Form and test a specific hypothesis. 5. Apply a supported corrective action. 6. Verify service recovery and document the RCA.

Senior interview Q&A

Q: Why can pod ping fail while Service HTTP works?

A: ICMP may be filtered and Service traffic follows protocol/port and endpoint rules; use appropriate TCP tests.

Follow-up interview challenge

Explain the expected command output, one false-positive interpretation, one security concern, and how you would validate the fix without introducing a second outage.

Chapter 09 — Services, Routes, Ingress and TLS

Concept and operating principle

ClusterIP selects endpoints using labels; NodePort exposes ports on nodes; LoadBalancer integrates external LB where supported. OpenShift Route exposes HTTP(S) through ingress controller. Edge, passthrough and re-encrypt TLS differ in where termination occurs. Check endpoint readiness before debugging routers.

Step-by-step verification commands

```
oc get svc,route,endpointslices -n APP_NS
oc describe route APP_ROUTE -n APP_NS
oc get ingresscontroller -n openshift-ingress-operator
oc logs -n openshift-ingress deploy/router-default --tail=100
```

How to interpret the checks

Run these commands from an authenticated installation or administration workstation unless SSH/node debugging is specified. Start with read-only checks; compare observed conditions, events and timestamps against expected state. Confirm namespace and resource names before changes. Collect logs and evidence before remediation.

Production troubleshooting scenario

Route returns 503: verify service selector, targetPort, endpoint readiness, router and backend logs.

Response workflow

1. Establish impact and blast radius. 2. Inspect operator/node/resource conditions. 3. Correlate events and logs with recent changes. 4. Form and test a specific hypothesis. 5. Apply a supported corrective action. 6. Verify service recovery and document the RCA.

Senior interview Q&A

Q: Difference between Route passthrough and edge?

A: Passthrough preserves TLS to backend; edge terminates TLS at router.

Follow-up interview challenge

Explain the expected command output, one false-positive interpretation, one security concern, and how you would validate the fix without introducing a second outage.

Chapter 10 — DNS and egress

Concept and operating principle

CoreDNS resolves service names; external DNS depends on upstream connectivity. Internal API DNS and wildcard app DNS must resolve to intended load balancers. NAT gateways or approved egress paths provide outbound access for private nodes. EgressIP and egress firewall capabilities require compatible platform configuration.

Step-by-step verification commands

```
oc get dns.operator/default -o yaml
oc get pods -n openshift-dns
oc run dns-test --rm -it --restart=Never --image=registry.access.redhat.com/ubi9/ubi-
minimal -- nslookup kubernetes.default.svc
```

How to interpret the checks

Run these commands from an authenticated installation or administration workstation unless SSH/node debugging is specified. Start with read-only checks; compare observed conditions, events and timestamps against expected state. Confirm namespace and resource names before changes. Collect logs and evidence before remediation.

Production troubleshooting scenario

Pods resolve internal names but not external: inspect CoreDNS forwarding, VPC DNS, NAT routes and egress policies.

Response workflow

1. Establish impact and blast radius. 2. Inspect operator/node/resource conditions. 3. Correlate events and logs with recent changes. 4. Form and test a specific hypothesis. 5. Apply a supported corrective action. 6. Verify service recovery and document the RCA.

Senior interview Q&A

Q: Why is api-int DNS critical?

A: Nodes use the internal API endpoint for bootstrap and ongoing cluster communication.

Follow-up interview challenge

Explain the expected command output, one false-positive interpretation, one security concern, and how you would validate the fix without introducing a second outage.

Chapter 11 — Storage CSI, EBS and EFS

Concept and operating principle

StorageClass provisions PVs via CSI drivers. PVC requests storage, PV binds it, and pods mount it. EBS is zonal block storage; placement and volume topology matter. EFS provides shared file access with a compatible CSI driver. Validate snapshots, encryption, resize and reclaim policy.

Step-by-step verification commands

```
oc get sc
oc get pvc,pv -A
oc get csidriver
oc describe pvc DATA_PVC -n APP_NS
oc get volumeattachments
```

How to interpret the checks

Run these commands from an authenticated installation or administration workstation unless SSH/node debugging is specified. Start with read-only checks; compare observed conditions, events and timestamps against expected state. Confirm namespace and resource names before changes. Collect logs and evidence before remediation.

Production troubleshooting scenario

PVC Pending: inspect StorageClass, CSI driver, AWS IAM, capacity, topology and PVC events.

Response workflow

1. Establish impact and blast radius. 2. Inspect operator/node/resource conditions. 3. Correlate events and logs with recent changes. 4. Form and test a specific hypothesis. 5. Apply a supported corrective action. 6. Verify service recovery and document the RCA.

Senior interview Q&A

Q: Why does EBS AZ mismatch matter?

A: An EBS volume attaches within its AZ; scheduling must respect PV topology.

Follow-up interview challenge

Explain the expected command output, one false-positive interpretation, one security concern, and how you would validate the fix without introducing a second outage.

Chapter 12 — Authentication, OAuth and RBAC

Concept and operating principle

Authentication identifies users; authorization controls actions. Integrate supported OAuth identity providers and define least-privilege Roles/ClusterRoles with bindings. Use service accounts for automation, avoid shared admin credentials, and periodically audit grants.

Step-by-step verification commands

```
oc get oauth cluster -o yaml
oc auth can-i create deployments -n APP_NS --as USER
oc get rolebinding,clusterrolebinding -A
oc adm policy who-can get secrets -n APP_NS
```

How to interpret the checks

Run these commands from an authenticated installation or administration workstation unless SSH/node debugging is specified. Start with read-only checks; compare observed conditions, events and timestamps against expected state. Confirm namespace and resource names before changes. Collect logs and evidence before remediation.

Production troubleshooting scenario

User can view pods but not secrets: inspect namespace RoleBindings and ClusterRoles, verify intended policy before granting additional access.

Response workflow

1. Establish impact and blast radius. 2. Inspect operator/node/resource conditions. 3. Correlate events and logs with recent changes. 4. Form and test a specific hypothesis. 5. Apply a supported corrective action. 6. Verify service recovery and document the RCA.

Senior interview Q&A

Q: Role vs ClusterRole?

A: Role is namespace scoped; ClusterRole defines permissions usable cluster-wide or through a namespace binding.

Follow-up interview challenge

Explain the expected command output, one false-positive interpretation, one security concern, and how you would validate the fix without introducing a second outage.

Chapter 13 — SCC and workload security

Concept and operating principle

Security Context Constraints restrict pod privileges, capabilities, host access, UID strategies and volumes. Use restricted defaults; assign elevated SCC only to approved service accounts when required. Pair SCC with RBAC, NetworkPolicy, image provenance and admission policies.

Step-by-step verification commands

```
oc get scc
oc describe scc restricted-v2
oc adm policy who-can use scc/privileged
oc get sa -n APP_NS
```

How to interpret the checks

Run these commands from an authenticated installation or administration workstation unless SSH/node debugging is specified. Start with read-only checks; compare observed conditions, events and timestamps against expected state. Confirm namespace and resource names before changes. Collect logs and evidence before remediation.

Production troubleshooting scenario

Pod rejected by SCC: inspect admission message, securityContext and allowed UID/capabilities; fix image compatibility instead of granting privileged SCC by default.

Response workflow

1. Establish impact and blast radius. 2. Inspect operator/node/resource conditions. 3. Correlate events and logs with recent changes. 4. Form and test a specific hypothesis. 5. Apply a supported corrective action. 6. Verify service recovery and document the RCA.

Senior interview Q&A

Q: Does RBAC replace SCC?

A: No. RBAC authorizes API actions; SCC constrains pod security at admission.

Follow-up interview challenge

Explain the expected command output, one false-positive interpretation, one security concern, and how you would validate the fix without introducing a second outage.

Chapter 14 — Secrets, certificates and audit

Concept and operating principle

Secrets store sensitive application data but are not automatically a complete secret-management solution. Use RBAC restrictions, etcd encryption policies, external secret integration and rotation. Manage API/ingress certificate lifecycle; inspect audit policy and secure log retention. Avoid printing secret values into CI logs.

Step-by-step verification commands

```
oc create secret generic app-db --from-literal=username=app --from-
literal=password=REPLACE -n APP_NS
oc get secret app-db -n APP_NS -o yaml
oc get apiserver cluster -o yaml
oc get kubeapiserver.operator.openshift.io cluster -o yaml
```

How to interpret the checks

Run these commands from an authenticated installation or administration workstation unless SSH/node debugging is specified. Start with read-only checks; compare observed conditions, events and timestamps against expected state. Confirm namespace and resource names before changes. Collect logs and evidence before remediation.

Production troubleshooting scenario

Expired ingress certificate: check route TLS mode, secret, chain, DNS and router rollout; rotate through supported workflow.

Response workflow

1. Establish impact and blast radius. 2. Inspect operator/node/resource conditions. 3. Correlate events and logs with recent changes. 4. Form and test a specific hypothesis. 5. Apply a supported corrective action. 6. Verify service recovery and document the RCA.

Senior interview Q&A

Q: Are Kubernetes Secrets encrypted by default?

A: Base64 encoding is not encryption; storage encryption must be configured and verified separately.

Follow-up interview challenge

Explain the expected command output, one false-positive interpretation, one security concern, and how you would validate the fix without introducing a second outage.

Chapter 15 — Monitoring, alerting and logs

Concept and operating principle

OpenShift monitoring uses Prometheus and Alertmanager with operator-managed components. Track SLOs, saturation, node pressure, API latency, etcd health and application error rates. Collect must-gather bundles and targeted logs with timestamps; redact secrets before sharing.

Step-by-step verification commands

```
oc get pods -n openshift-monitoring
oc get alertmanager -n openshift-monitoring
oc adm must-gather --dest-dir=./must-gather
oc logs deployment/APP -n APP_NS --since=30m
```

How to interpret the checks

Run these commands from an authenticated installation or administration workstation unless SSH/node debugging is specified. Start with read-only checks; compare observed conditions, events and timestamps against expected state. Confirm namespace and resource names before changes. Collect logs and evidence before remediation.

Production troubleshooting scenario

API latency spike: correlate etcd metrics, control-plane resource usage, request rates, network and recent changes.

Response workflow

1. Establish impact and blast radius. 2. Inspect operator/node/resource conditions. 3. Correlate events and logs with recent changes. 4. Form and test a specific hypothesis. 5. Apply a supported corrective action. 6. Verify service recovery and document the RCA.

Senior interview Q&A

Q: What is must-gather?

A: A supported diagnostic collection that captures cluster resources and operator information for analysis.

Follow-up interview challenge

Explain the expected command output, one false-positive interpretation, one security concern, and how you would validate the fix without introducing a second outage.

Chapter 16 — Node operations and scheduling

Concept and operating principle

Day-2 tasks include cordon/drain/uncordon, taints/tolerations, affinities, topology spread, quotas and limit ranges. Drain honors eviction/PDB constraints. Always verify workload replica distribution and capacity before planned node maintenance.

Step-by-step verification commands

```
oc adm cordon NODE
oc adm drain NODE --ignore-daemonsets --delete-emptydir-data
oc adm uncordon NODE
oc get pdb -A
oc get resourcequota,limitrange -A
```

How to interpret the checks

Run these commands from an authenticated installation or administration workstation unless SSH/node debugging is specified. Start with read-only checks; compare observed conditions, events and timestamps against expected state. Confirm namespace and resource names before changes. Collect logs and evidence before remediation.

Production troubleshooting scenario

Node maintenance blocked: inspect PDB, unhealthy workloads, local data and available replicas; do not bypass safeguards without approved impact analysis.

Response workflow

1. Establish impact and blast radius. 2. Inspect operator/node/resource conditions. 3. Correlate events and logs with recent changes. 4. Form and test a specific hypothesis. 5. Apply a supported corrective action. 6. Verify service recovery and document the RCA.

Senior interview Q&A

Q: Why use topologySpreadConstraints?

A: To distribute replicas across failure domains and reduce correlated downtime.

Follow-up interview challenge

Explain the expected command output, one false-positive interpretation, one security concern, and how you would validate the fix without introducing a second outage.

Chapter 17 — Capacity, performance and autoscaling

Concept and operating principle

Requests drive scheduling; limits constrain usage and may trigger OOMKill or CPU throttling. Use metrics to evaluate memory pressure, disk I/O, CPU saturation and network. HPA scales replicas from supported metrics; cluster autoscaling requires supported Machine API and infrastructure setup.

Step-by-step verification commands

```
oc adm top nodes
oc adm top pods -A
oc describe pod POD -n APP_NS
oc get hpa -A
oc get events -A --sort-by=.lastTimestamp
```

How to interpret the checks

Run these commands from an authenticated installation or administration workstation unless SSH/node debugging is specified. Start with read-only checks; compare observed conditions, events and timestamps against expected state. Confirm namespace and resource names before changes. Collect logs and evidence before remediation.

Production troubleshooting scenario

Pod OOMKilled: check last termination, memory graphs, workload leaks, requests/limits and right-sizing.

Response workflow

1. Establish impact and blast radius. 2. Inspect operator/node/resource conditions. 3. Correlate events and logs with recent changes. 4. Form and test a specific hypothesis. 5. Apply a supported corrective action. 6. Verify service recovery and document the RCA.

Senior interview Q&A

Q: Why is high load average not identical to CPU utilization?

A: Load average includes runnable and some uninterruptible tasks; CPU percent measures processor activity.

Follow-up interview challenge

Explain the expected command output, one false-positive interpretation, one security concern, and how you would validate the fix without introducing a second outage.

Chapter 18 — Application deployment, YAML and rollout

Concept and operating principle

Deployments manage ReplicaSets and rollout history. ConfigMaps contain nonsecret configuration; Secrets provide sensitive inputs. Services use label selectors; Routes expose service. Validate readiness/liveness/startup probes and use image digests for repeatability.

Step-by-step verification commands

```
oc new-project sample-app
oc apply -f deployment.yaml
oc apply -f service.yaml
oc apply -f route.yaml
oc rollout status deployment/web -n sample-app
oc rollout undo deployment/web -n sample-app
```

How to interpret the checks

Run these commands from an authenticated installation or administration workstation unless SSH/node debugging is specified. Start with read-only checks; compare observed conditions, events and timestamps against expected state. Confirm namespace and resource names before changes. Collect logs and evidence before remediation.

Production troubleshooting scenario

Deployment pods Running but Service unavailable: compare selector to pod labels, port to targetPort and readiness/endpoints.

Response workflow

1. Establish impact and blast radius. 2. Inspect operator/node/resource conditions. 3. Correlate events and logs with recent changes. 4. Form and test a specific hypothesis. 5. Apply a supported corrective action. 6. Verify service recovery and document the RCA.

Senior interview Q&A

Q: What does readiness probe control?

A: Whether a pod is eligible to receive Service traffic.

Follow-up interview challenge

Explain the expected command output, one false-positive interpretation, one security concern, and how you would validate the fix without introducing a second outage.

Chapter 19 — Jenkins, CI/CD and GitOps

Concept and operating principle

Jenkins pipeline should lint/test, scan dependencies and images, build immutable artifacts, push to trusted registry and deploy with least-privilege service account. Prefer short-lived tokens and controlled credential storage. Argo CD reconciles Git desired state and detects drift; Helm templates reusable Kubernetes manifests.

Step-by-step verification commands

```
oc create sa jenkins-deployer -n sample-app
oc adm policy add-role-to-user edit -z jenkins-deployer -n sample-app
oc rollout status deployment/web -n sample-app
oc get applications.argoproj.io -A
```

How to interpret the checks

Run these commands from an authenticated installation or administration workstation unless SSH/node debugging is specified. Start with read-only checks; compare observed conditions, events and timestamps against expected state. Confirm namespace and resource names before changes. Collect logs and evidence before remediation.

Production troubleshooting scenario

Jenkins deployment fails Forbidden: check service account identity, namespace RoleBindings and token scope; avoid granting cluster-admin.

Response workflow

1. Establish impact and blast radius. 2. Inspect operator/node/resource conditions. 3. Correlate events and logs with recent changes. 4. Form and test a specific hypothesis. 5. Apply a supported corrective action. 6. Verify service recovery and document the RCA.

Senior interview Q&A

Q: What is the difference between CI and GitOps CD?

A: CI builds/tests artifacts; GitOps CD continuously reconciles cluster state from version-controlled manifests.

Follow-up interview challenge

Explain the expected command output, one false-positive interpretation, one security concern, and how you would validate the fix without introducing a second outage.

Chapter 20 — Disconnected clusters and registry

Concept and operating principle

Disconnected operations require mirrored release images, operator catalogs and application images in trusted registries, with image digest mappings, pull secrets, trust bundles and restricted egress. Test upgrades and catalog compatibility offline. Avoid assuming internet access during troubleshooting.

Step-by-step verification commands

```
oc get imagedigestmirrorsets
oc get imagecontentsourcepolicies
oc get catalogsources -A
oc get proxy cluster -o yaml
```

How to interpret the checks

Run these commands from an authenticated installation or administration workstation unless SSH/node debugging is specified. Start with read-only checks; compare observed conditions, events and timestamps against expected state. Confirm namespace and resource names before changes. Collect logs and evidence before remediation.

Production troubleshooting scenario

ImagePullBackOff offline: inspect image references, mirror mappings, registry DNS/TLS, pull credentials and catalog availability.

Response workflow

1. Establish impact and blast radius. 2. Inspect operator/node/resource conditions. 3. Correlate events and logs with recent changes. 4. Form and test a specific hypothesis. 5. Apply a supported corrective action. 6. Verify service recovery and document the RCA.

Senior interview Q&A

Q: Why mirror by digest?

A: Digest pins immutable content and reduces ambiguity from mutable image tags.

Follow-up interview challenge

Explain the expected command output, one false-positive interpretation, one security concern, and how you would validate the fix without introducing a second outage.

Chapter 21 — HA, backups, OADP and DR

Concept and operating principle

Separate etcd snapshots from application backup. OADP can protect supported Kubernetes resources and persistent volumes; test restore to a clean environment. Define RPO/RTO, failure-domain dependencies, external registry availability and disaster communication. Multi-AZ alone is not a DR plan.

Step-by-step verification commands

```
oc get co
oc get nodes -o wide
oc get backups.velero.io -A
oc get restores.velero.io -A
oc get volumesnapshot -A
```

How to interpret the checks

Run these commands from an authenticated installation or administration workstation unless SSH/node debugging is specified. Start with read-only checks; compare observed conditions, events and timestamps against expected state. Confirm namespace and resource names before changes. Collect logs and evidence before remediation.

Production troubleshooting scenario

AZ failure: verify etcd quorum, surviving ingress endpoints, worker capacity, PV zone constraints and restoration priorities.

Response workflow

1. Establish impact and blast radius. 2. Inspect operator/node/resource conditions. 3. Correlate events and logs with recent changes. 4. Form and test a specific hypothesis. 5. Apply a supported corrective action. 6. Verify service recovery and document the RCA.

Senior interview Q&A

Q: Difference between etcd snapshot and OADP backup?

A: etcd protects cluster control-plane state; OADP targets selected application resources and supported volume data.

Follow-up interview challenge

Explain the expected command output, one false-positive interpretation, one security concern, and how you would validate the fix without introducing a second outage.

Chapter 22 — Incidents, RCA and senior operations

Concept and operating principle

Senior admins own incident triage, change review, severity classification, evidence collection, recovery verification and postmortems. Build runbooks with trigger, impact, checks, rollback/forward fix, escalation and communication. Never run destructive recovery commands without a verified plan.

Step-by-step verification commands

```
oc get co,nodes
oc get events -A --sort-by=.lastTimestamp
oc adm must-gather
aws cloudformation describe-stack-events --stack-name STACK
```

How to interpret the checks

Run these commands from an authenticated installation or administration workstation unless SSH/node debugging is specified. Start with read-only checks; compare observed conditions, events and timestamps against expected state. Confirm namespace and resource names before changes. Collect logs and evidence before remediation.

Production troubleshooting scenario

Multiple degraded operators: establish common dependencies (API, DNS, network, storage, certificates), identify earliest failure and recent changes before fixing symptoms.

Response workflow

1. Establish impact and blast radius. 2. Inspect operator/node/resource conditions. 3. Correlate events and logs with recent changes. 4. Form and test a specific hypothesis. 5. Apply a supported corrective action. 6. Verify service recovery and document the RCA.

Senior interview Q&A

Q: What makes a good RCA?

A: Timeline, impact, evidence-backed root cause, contributing factors, corrective actions and preventive controls.

Follow-up interview challenge

Explain the expected command output, one false-positive interpretation, one security concern, and how you would validate the fix without introducing a second outage.

Appendix A — Deployments, Services and Routes

Example manifests. Replace image with a pinned, scanned digest for production. Check the container runs under the assigned SCC and listens on the expected port.

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: web
  namespace: sample-app
spec:
  replicas: 3
  selector:
    matchLabels: {app: web}
  template:
    metadata:
      labels: {app: web}
    spec:
      containers:
      - name: web
        image: registry.access.redhat.com/ubi9/httpd-24:latest
        ports: [{containerPort: 8080}]
        readinessProbe:
          tcpSocket: {port: 8080}
          initialDelaySeconds: 5
          periodSeconds: 10
        resources:
          requests: {cpu: 100m, memory: 128Mi}
          limits: {cpu: 500m, memory: 512Mi}
---
apiVersion: v1
kind: Service
metadata: {name: web, namespace: sample-app}
spec:
  selector: {app: web}
  ports: [{name: http, port: 80, targetPort: 8080}]
---
apiVersion: route.openshift.io/v1
kind: Route
metadata: {name: web, namespace: sample-app}
spec:
  to: {kind: Service, name: web}
  port: {targetPort: http}
  tls: {termination: edge, insecureEdgeTerminationPolicy: Redirect}
```

YAML line-by-line reasoning

Deployment.spec.replicas requests three pod replicas; selector.matchLabels must match template.metadata.labels. Container port documents the listening port; Service.spec.selector selects ready pods; targetPort maps external Service port 80 to container port 8080. Route.spec.to points at the Service, and edge TLS terminates HTTPS at the router. Readiness prevents unready endpoints from receiving traffic.

Secret creation and mounting

```
oc -n sample-app create secret generic db-credentials --from-literal=username=app --
from-literal=password=REPLACE
oc -n sample-app get secret db-credentials
oc -n sample-app set env deployment/web --from=secret/db-credentials
oc -n sample-app rollout status deployment/web
```

Never put real credentials in commands saved to shell history. Prefer approved secret-management integrations or secure input files; rotate exposed values immediately.

Appendix B — Jenkins deployment example

Illustrative pipeline skeleton; it does not perform real tests or image scanning until you replace the placeholder stage. The deploy identity must be namespace-scoped and its token must be managed securely.

```
pipeline {
  agent any
  stages {
    stage('Test') { steps { sh 'echo run tests and SAST here' } }
    stage('Deploy') {
      steps {
        withCredentials([string(credentialsId: 'openshift-token', variable: 'TOKEN')]) {
          sh '''
            set +x
            oc login "$OCP_API" --token="$TOKEN"
            oc -n sample-app apply -f k8s/
            oc -n sample-app rollout status deployment/web --timeout=5m
          '''
        }
      }
    }
  }
}
```

Production extensions

Add source validation, unit/integration tests, SonarQube or equivalent SAST, dependency and container scanning, signed image publishing, approval gates, immutable digests, deployment health checks, progressive delivery and a documented rollback strategy. Do not expose secrets in pipeline logs.

Appendix C — Glossary

API server — Kubernetes control-plane API entry point

CVO — Cluster Version Operator; reconciles OpenShift release components

MCO — Machine Config Operator; manages RHCOS configuration

MCP — MachineConfigPool; group of nodes sharing rendered config

MCD — Machine Config Daemon running on nodes

etcd — Distributed strongly consistent control-plane state store

Ignition — RHCOS first-boot provisioning configuration

RHCOS — Red Hat Enterprise Linux CoreOS

OVN-Kubernetes — OpenShift default software-defined network implementation

SCC — Security Context Constraints admission mechanism

RBAC — Role-Based Access Control

CSI — Container Storage Interface

PV/PVC — PersistentVolume and PersistentVolumeClaim

PDB — PodDisruptionBudget

OADP — OpenShift API for Data Protection

Route — OpenShift resource for HTTP(S) ingress

STS — AWS Security Token Service temporary credentials

UPI — User-Provisioned Infrastructure

RPO — Recovery Point Objective; acceptable data loss window

RTO — Recovery Time Objective; target service restoration time

OLM — Operator Lifecycle Manager

SLO — Service Level Objective

Appendix D — Production readiness checklist

☐ Version-matched RHCOS image and verified AWS template schemas

☐ Multi-AZ control plane, workers, LBs and NAT design

☐ Private API/MCS reachability and correct DNS records

☐ IAM least privilege, short-lived credentials and audit logging

☐ Cluster operators healthy and nodes Ready

☐ MCP healthy; tested update/maintenance procedures

☐ TLS certificate ownership and renewal monitoring

☐ CSI storage, encrypted backups and restore drill

☐ etcd snapshot procedure and off-cluster retention

☐ Application PDBs, probes, quotas and network policies

☐ Monitoring, alert routing, log retention and incident runbooks

☐ Tested Jenkins/GitOps service-account permissions

☐ Cost, capacity and disaster recovery exercises

Reference documentation

Red Hat OpenShift Container Platform 4.20 documentation:
https://docs.redhat.com/en/documentation/openshift_container_platform/4.20/

AWS CloudFormation documentation: https://docs.aws.amazon.com/AWSCloudFormation/latest/UserGuide/Welcome.html

OPENSHIFT 4.20

AWS UPI Deployment + Senior Administrator Deep-Dive Handbook

Bilingual Hinglish concepts | operational runbooks | YAML | troubleshooting | interview Q&A

Release-aware study edition • 16 October 2026 • Technical examples are illustrative, not evidence of a deployed cluster.

IMPORTANT: CloudFormation template parameters, AMIs, IAM mode and recovery actions must be matched to the exact OpenShift 4.20 z-stream and Red Hat documentation. No production AWS account was accessed.

Contents and study method

01. AWS UPI release planning

02. AWS IAM and STS

03. VPC and DNS

04. Load balancers and security

05. Installer and Ignition

06. Bootstrap and control plane

07. Workers and CSR

08. OpenShift control plane internals

09. RHCOS and MachineConfig

10. Projects and administration

11. Pods, Deployments and YAML

12. Services, Routes and DNS

13. OVN-Kubernetes networking

14. Secrets, RBAC and SCC

15. Storage and CSI

16. Operators and OLM

17. Updates and CVO

18. Monitoring and logging

19. etcd and disaster recovery

20. Jenkins and GitOps

21. Production troubleshooting

22. Senior interview simulation

Reading method: Each chapter explains the principle, commands, verification, failures and senior-level interview response. Example commands are starting points; validate against live objects and documentation.

01. AWS UPI release planning

Concept and internal working | Principle

UPI mein installer cluster assets generate karta hai, lekin AWS infrastructure ka lifecycle administrator own karta hai. Inventory VPC, IAM, quotas, RHCOS AMIs, DNS, NLB target groups, availability zones, cost, TLS and support contract before any stack.

Production design and safe operating workflow

Step 1 — Identify desired state and dependencies. Step 2 — Verify authentication, release, permissions and prerequisites. Step 3 — Apply only approved configuration in a test environment. Step 4 — Inspect controller reconciliation and component logs. Step 5 — Validate functionality and document rollback or recovery plan. Hinglish: sirf command chalana enough nahi; command ke peeche control loop aur failure domain samajhna zaroori hai.

Hands-on lab: AWS UPI release planning

Objective and prerequisites: Run these commands on an authorized OpenShift 4.20 lab using an identity with the required permissions. Never use destructive commands against production without change approval.

```
openshift-install version
aws sts get-caller-identity
aws ec2 describe-availability-zones --region ap-south-1
aws service-quotas list-service-quotas --service-code ec2
```

Expected verification: Version, region and account match design. Verify quotas for 3 masters, 3 workers, temporary bootstrap and load balancers.

Failure investigation: If installation fails before bootstrap, inspect AWS permissions, service quotas, CloudFormation events and regional AMI support.

Interview Q: How is UPI different from IPI?

Answer: UPI operator owns provisioning and ongoing infrastructure lifecycle; IPI installer creates supported AWS infrastructure. Both use the installer to generate OpenShift assets.

Scenario drill — senior engineer reasoning

Situation: This component becomes unhealthy immediately after a scheduled infrastructure change. First determine blast radius, inspect timestamps and last known-good state. Gather read-only evidence, check upstream dependencies, and propose the lowest-risk corrective action. State which signals would confirm the hypothesis and how you would validate recovery. Avoid guessing a root cause.

Documentation checkpoint: verify version-specific APIs, flags, supported recovery actions and release caveats in the official Red Hat 4.20 manuals before production execution.

02. AWS IAM and STS

Concept and internal working | Principle

AWS permissions split into installation identity, machine instance profiles and cluster components. Least privilege aur credential lifecycle senior-level design decisions hain. STS short-lived credentials and operator role mappings must follow supported release procedures; do not assume workstation SSO credentials replace cluster credential configuration.

Production design and safe operating workflow

Step 1 — Identify desired state and dependencies. Step 2 — Verify authentication, release, permissions and prerequisites. Step 3 — Apply only approved configuration in a test environment. Step 4 — Inspect controller reconciliation and component logs. Step 5 — Validate functionality and document rollback or recovery plan. Hinglish: sirf command chalana enough nahi; command ke peeche control loop aur failure domain samajhna zaroori hai.

Hands-on lab: AWS IAM and STS

Objective and prerequisites: Run these commands on an authorized OpenShift 4.20 lab using an identity with the required permissions. Never use destructive commands against production without change approval.

```
aws iam list-instance-profiles --query "InstanceProfiles[].InstanceProfileName"
aws sts get-caller-identity
aws cloudformation describe-stack-events --stack-name ocp-prod-security
```

Expected verification: Instance profiles exist and role trust policies match the intended AWS services.

Failure investigation: AccessDenied: locate the denied AWS API action in CloudTrail; check role trust, policy, permissions boundary, SCP and session restrictions.

Interview Q: Why can AWS CLI work but cloud-controller fail?

Answer: The CLI and cluster workloads use different identities and policies; check instance profiles, credentials mode, STS trust, service account roles and CloudTrail.

Scenario drill — senior engineer reasoning

Situation: This component becomes unhealthy immediately after a scheduled infrastructure change. First determine blast radius, inspect timestamps and last known-good state. Gather read-only evidence, check upstream dependencies, and propose the lowest-risk corrective action. State which signals would confirm the hypothesis and how you would validate recovery. Avoid guessing a root cause.

Documentation checkpoint: verify version-specific APIs, flags, supported recovery actions and release caveats in the official Red Hat 4.20 manuals before production execution.

03. VPC and DNS

Concept and internal working | Principle

3 AZ design with 3 private node subnets, public LB/NAT subnets, Route 53 public and private zones. api external resolves public LB; api-int internal resolves private LB. CIDR overlap with pod/service ranges breaks routing. NAT or VPC endpoints must satisfy outbound dependencies.

Production design and safe operating workflow

Step 1 — Identify desired state and dependencies. Step 2 — Verify authentication, release, permissions and prerequisites. Step 3 — Apply only approved configuration in a test environment. Step 4 — Inspect controller reconciliation and component logs. Step 5 — Validate functionality and document rollback or recovery plan. Hinglish: sirf command chalana enough nahi; command ke peeche control loop aur failure domain samajhna zaroori hai.

Hands-on lab: VPC and DNS

Objective and prerequisites: Run these commands on an authorized OpenShift 4.20 lab using an identity with the required permissions. Never use destructive commands against production without change approval.

```
aws ec2 describe-vpcs
aws ec2 describe-subnets --filters Name=vpc-id,Values=$VPC_ID
aws ec2 describe-route-tables --filters Name=vpc-id,Values=$VPC_ID
dig +short api.ocp-prod.example.com
dig +short api-int.ocp-prod.example.com
```

Expected verification: Public subnets route to IGW; private node subnets have intended egress and VPC DNS works.

Failure investigation: DNS timeout: inspect private hosted zone association, DHCP options, VPC resolver, security groups and upstream forwarding.

Interview Q: Why is api-int important?

Answer: Internal API and machine-config traffic must resolve and route within the VPC, independent of public DNS.

Scenario drill — senior engineer reasoning

Situation: This component becomes unhealthy immediately after a scheduled infrastructure change. First determine blast radius, inspect timestamps and last known-good state. Gather read-only evidence, check upstream dependencies, and propose the lowest-risk corrective action. State which signals would confirm the hypothesis and how you would validate recovery. Avoid guessing a root cause.

Documentation checkpoint: verify version-specific APIs, flags, supported recovery actions and release caveats in the official Red Hat 4.20 manuals before production execution.

04. Load balancers and security

Concept and internal working | Principle

External API TCP/6443, internal API TCP/6443 and MCS TCP/22623, ingress 80/443. During bootstrap, target groups temporarily include bootstrap plus masters; remove bootstrap only after successful handoff. MCS must not be public. Security group rules must also support etcd, kubelet and OVN.

Production design and safe operating workflow

Step 1 — Identify desired state and dependencies. Step 2 — Verify authentication, release, permissions and prerequisites. Step 3 — Apply only approved configuration in a test environment. Step 4 — Inspect controller reconciliation and component logs. Step 5 — Validate functionality and document rollback or recovery plan. Hinglish: sirf command chalana enough nahi; command ke peeche control loop aur failure domain samajhna zaroori hai.

Hands-on lab: Load balancers and security

Objective and prerequisites: Run these commands on an authorized OpenShift 4.20 lab using an identity with the required permissions. Never use destructive commands against production without change approval.

```
aws elbv2 describe-load-balancers
aws elbv2 describe-target-groups
aws elbv2 describe-target-health --target-group-arn "$API_TG"
aws ec2 describe-security-groups --group-ids "$MASTER_SG"
```

Expected verification: LB targets healthy after services start; 22623 remains internal-only.

Failure investigation: Target unhealthy: verify TCP reachability, target IP/instance registration, SG/NACL, listener, target group port and node process.

Interview Q: How do you distinguish API LB failure from etcd failure?

Answer: Check DNS and TCP reachability, LB target health, kube-apiserver static pods and etcd endpoint health; do not assume one symptom implies one root cause.

Scenario drill — senior engineer reasoning

Situation: This component becomes unhealthy immediately after a scheduled infrastructure change. First determine blast radius, inspect timestamps and last known-good state. Gather read-only evidence, check upstream dependencies, and propose the lowest-risk corrective action. State which signals would confirm the hypothesis and how you would validate recovery. Avoid guessing a root cause.

Documentation checkpoint: verify version-specific APIs, flags, supported recovery actions and release caveats in the official Red Hat 4.20 manuals before production execution.

05. Installer and Ignition

Concept and internal working | Principle

install-config.yaml is source configuration. create manifests materializes cluster manifests; create ignition-configs emits bootstrap/master/worker Ignition and metadata. Ignition executes at first boot and configures systemd/files/certs. Store assets securely and track expiration.

Production design and safe operating workflow

Step 1 — Identify desired state and dependencies. Step 2 — Verify authentication, release, permissions and prerequisites. Step 3 — Apply only approved configuration in a test environment. Step 4 — Inspect controller reconciliation and component logs. Step 5 — Validate functionality and document rollback or recovery plan. Hinglish: sirf command chalana enough nahi; command ke peeche control loop aur failure domain samajhna zaroori hai.

Hands-on lab: Installer and Ignition

Objective and prerequisites: Run these commands on an authorized OpenShift 4.20 lab using an identity with the required permissions. Never use destructive commands against production without change approval.

```
openshift-install create manifests --dir "$INSTALL_DIR"
openshift-install create ignition-configs --dir "$INSTALL_DIR"
jq -r .infraID "$INSTALL_DIR/metadata.json"
ls -l "$INSTALL_DIR"/*.ign
```

Expected verification: Expected bootstrap.ign, master.ign, worker.ign and metadata.json.

Failure investigation: First-boot failure: inspect console output and journal for Ignition URL, TLS, auth and malformed configuration.

Interview Q: Why cannot you casually regenerate Ignition after nodes start?

Answer: Ignition is first-boot state and generated assets tie to a particular cluster identity and certificate timeline; regeneration can invalidate assumptions.

Scenario drill — senior engineer reasoning

Situation: This component becomes unhealthy immediately after a scheduled infrastructure change. First determine blast radius, inspect timestamps and last known-good state. Gather read-only evidence, check upstream dependencies, and propose the lowest-risk corrective action. State which signals would confirm the hypothesis and how you would validate recovery. Avoid guessing a root cause.

Documentation checkpoint: verify version-specific APIs, flags, supported recovery actions and release caveats in the official Red Hat 4.20 manuals before production execution.

06. Bootstrap and control plane

Concept and internal working | Principle

Temporary bootstrap starts early control plane; masters take ownership after etcd and API converge. 3 etcd voting members require 2 for quorum. Do not delete bootstrap before wait-for bootstrap-complete succeeds.

Production design and safe operating workflow

Step 1 — Identify desired state and dependencies. Step 2 — Verify authentication, release, permissions and prerequisites. Step 3 — Apply only approved configuration in a test environment. Step 4 — Inspect controller reconciliation and component logs. Step 5 — Validate functionality and document rollback or recovery plan. Hinglish: sirf command chalana enough nahi; command ke peeche control loop aur failure domain samajhna zaroori hai.

Hands-on lab: Bootstrap and control plane

Objective and prerequisites: Run these commands on an authorized OpenShift 4.20 lab using an identity with the required permissions. Never use destructive commands against production without change approval.

```
openshift-install wait-for bootstrap-complete --dir "$INSTALL_DIR" --log-level=debug
openshift-install gather bootstrap --dir "$INSTALL_DIR"
```

Expected verification: Successful bootstrap completion explicitly indicates temporary resources can be removed.

Failure investigation: Stall: check API/MCS target registration, RHCOS Ignition, DNS, etcd member health, certificates and CloudFormation stack events.

Interview Q: What does bootstrap complete prove?

Answer: It proves handoff of bootstrap functions, not that every cluster operator or application workload is healthy.

Scenario drill — senior engineer reasoning

Situation: This component becomes unhealthy immediately after a scheduled infrastructure change. First determine blast radius, inspect timestamps and last known-good state. Gather read-only evidence, check upstream dependencies, and propose the lowest-risk corrective action. State which signals would confirm the hypothesis and how you would validate recovery. Avoid guessing a root cause.

Documentation checkpoint: verify version-specific APIs, flags, supported recovery actions and release caveats in the official Red Hat 4.20 manuals before production execution.

07. Workers and CSR

Concept and internal working | Principle

UPI workers boot with worker Ignition and join kubelet TLS bootstrap. CSRs should be inspected for subject, signer and intended node identity. Approve only legitimate requests. Verify taints, allocatable capacity and node labels.

Production design and safe operating workflow

Step 1 — Identify desired state and dependencies. Step 2 — Verify authentication, release, permissions and prerequisites. Step 3 — Apply only approved configuration in a test environment. Step 4 — Inspect controller reconciliation and component logs. Step 5 — Validate functionality and document rollback or recovery plan. Hinglish: sirf command chalana enough nahi; command ke peeche control loop aur failure domain samajhna zaroori hai.

Hands-on lab: Workers and CSR

Objective and prerequisites: Run these commands on an authorized OpenShift 4.20 lab using an identity with the required permissions. Never use destructive commands against production without change approval.

```
oc get csr
oc describe csr <csr-name>
oc adm certificate approve <verified-csr>
oc get nodes -o wide
oc describe node <worker>
```

Expected verification: Expected three worker nodes Ready after certificate approval and CNI readiness.

Failure investigation: Pending CSR: inspect requester identity and serving/client certificate flow; NotReady: kubelet logs, network, CRI-O and MCO.

Interview Q: Why not approve all CSRs?

Answer: Unverified CSR approval can issue credentials to unauthorized nodes or identities, undermining cluster trust.

Scenario drill — senior engineer reasoning

Situation: This component becomes unhealthy immediately after a scheduled infrastructure change. First determine blast radius, inspect timestamps and last known-good state. Gather read-only evidence, check upstream dependencies, and propose the lowest-risk corrective action. State which signals would confirm the hypothesis and how you would validate recovery. Avoid guessing a root cause.

Documentation checkpoint: verify version-specific APIs, flags, supported recovery actions and release caveats in the official Red Hat 4.20 manuals before production execution.

08. OpenShift control plane internals

Concept and internal working | Principle

API server validates and stores desired state in etcd. Controllers reconcile actual state. Scheduler assigns unscheduled pods. Kubelet and CRI-O realize pod state. CVO manages release payload; Operators own components. Static pods run critical control plane services.

Production design and safe operating workflow

Step 1 — Identify desired state and dependencies. Step 2 — Verify authentication, release, permissions and prerequisites. Step 3 — Apply only approved configuration in a test environment. Step 4 — Inspect controller reconciliation and component logs. Step 5 — Validate functionality and document rollback or recovery plan. Hinglish: sirf command chalana enough nahi; command ke peeche control loop aur failure domain samajhna zaroori hai.

Hands-on lab: OpenShift control plane internals

Objective and prerequisites: Run these commands on an authorized OpenShift 4.20 lab using an identity with the required permissions. Never use destructive commands against production without change approval.

```
oc get co
oc get clusterversion
oc get pods -n openshift-kube-apiserver
oc get pods -n openshift-etcd
oc get events -A --sort-by=.lastTimestamp
```

Expected verification: Operators Available, not Progressing or Degraded at steady state.

Failure investigation: If API unavailable, use node SSH and crictl/journal for static pod diagnosis; never rely only on oc.

Interview Q: How does desired-state reconciliation work?

Answer: Controllers watch API objects, compare desired and observed state, and make idempotent changes until conditions converge.

Scenario drill — senior engineer reasoning

Situation: This component becomes unhealthy immediately after a scheduled infrastructure change. First determine blast radius, inspect timestamps and last known-good state. Gather read-only evidence, check upstream dependencies, and propose the lowest-risk corrective action. State which signals would confirm the hypothesis and how you would validate recovery. Avoid guessing a root cause.

Documentation checkpoint: verify version-specific APIs, flags, supported recovery actions and release caveats in the official Red Hat 4.20 manuals before production execution.

09. RHCOS and MachineConfig

Concept and internal working | Principle

MachineConfig defines host-level changes; MCO renders config and MCD applies changes to nodes in a MachineConfigPool. Rollout may drain and reboot nodes. PDB, failed drain, invalid config and degraded daemon are common blockers.

Production design and safe operating workflow

Step 1 — Identify desired state and dependencies. Step 2 — Verify authentication, release, permissions and prerequisites. Step 3 — Apply only approved configuration in a test environment. Step 4 — Inspect controller reconciliation and component logs. Step 5 — Validate functionality and document rollback or recovery plan. Hinglish: sirf command chalana enough nahi; command ke peeche control loop aur failure domain samajhna zaroori hai.

Hands-on lab: RHCOS and MachineConfig

Objective and prerequisites: Run these commands on an authorized OpenShift 4.20 lab using an identity with the required permissions. Never use destructive commands against production without change approval.

```
oc get mcp
oc describe mcp worker
oc get mc
oc get nodes -l node-role.kubernetes.io/worker
oc logs -n openshift-machine-config-operator daemonset/machine-config-daemon --all-containers
--tail=80
```

Expected verification: MCP Updated=True and Degraded=False after controlled rollout.

Failure investigation: Stuck Updating: compare desired/current rendered config annotations, node status, PDB eviction and MCD logs.

Interview Q: Why can a harmless config change disrupt workloads?

Answer: MCO may drain and reboot nodes; insufficient replicas, restrictive PDBs and low spare capacity can block or interrupt rollout.

Scenario drill — senior engineer reasoning

Situation: This component becomes unhealthy immediately after a scheduled infrastructure change. First determine blast radius, inspect timestamps and last known-good state. Gather read-only evidence, check upstream dependencies, and propose the lowest-risk corrective action. State which signals would confirm the hypothesis and how you would validate recovery. Avoid guessing a root cause.

Documentation checkpoint: verify version-specific APIs, flags, supported recovery actions and release caveats in the official Red Hat 4.20 manuals before production execution.

10. Projects and administration

Concept and internal working | Principle

Projects isolate namespaced workloads, not necessarily network traffic. Admins manage quotas, LimitRanges, node labels, taints, maintenance, service accounts, roles and audit trails. Always scope oc commands explicitly.

Production design and safe operating workflow

Step 1 — Identify desired state and dependencies. Step 2 — Verify authentication, release, permissions and prerequisites. Step 3 — Apply only approved configuration in a test environment. Step 4 — Inspect controller reconciliation and component logs. Step 5 — Validate functionality and document rollback or recovery plan. Hinglish: sirf command chalana enough nahi; command ke peeche control loop aur failure domain samajhna zaroori hai.

Hands-on lab: Projects and administration

Objective and prerequisites: Run these commands on an authorized OpenShift 4.20 lab using an identity with the required permissions. Never use destructive commands against production without change approval.

```
oc new-project payments
oc get resourcequota,limitrange -n payments
oc adm top nodes
oc adm cordon <node>
oc adm drain <node> --ignore-daemonsets --delete-emptydir-data
```

Expected verification: Before drain, validate replicas, PDB and workload storage; after maintenance uncordon.

Failure investigation: Eviction blocked: inspect PDB and capacity; do not force delete production pods without impact assessment.

Interview Q: How do you safely maintain a worker?

Answer: Check alerts, workload redundancy, PDB, storage attachments and capacity; cordon, drain, perform maintenance, verify, uncordon.

Scenario drill — senior engineer reasoning

Situation: This component becomes unhealthy immediately after a scheduled infrastructure change. First determine blast radius, inspect timestamps and last known-good state. Gather read-only evidence, check upstream dependencies, and propose the lowest-risk corrective action. State which signals would confirm the hypothesis and how you would validate recovery. Avoid guessing a root cause.

Documentation checkpoint: verify version-specific APIs, flags, supported recovery actions and release caveats in the official Red Hat 4.20 manuals before production execution.

11. Pods, Deployments and YAML

Concept and internal working | Principle

Deployment manages ReplicaSets and rolling updates. Readiness determines service endpoint eligibility; liveness restarts unhealthy containers; startup protects slow boot. Requests drive scheduling and limits constrain resource usage.

Production design and safe operating workflow

Step 1 — Identify desired state and dependencies. Step 2 — Verify authentication, release, permissions and prerequisites. Step 3 — Apply only approved configuration in a test environment. Step 4 — Inspect controller reconciliation and component logs. Step 5 — Validate functionality and document rollback or recovery plan. Hinglish: sirf command chalana enough nahi; command ke peeche control loop aur failure domain samajhna zaroori hai.

Hands-on lab: Pods, Deployments and YAML

Objective and prerequisites: Run these commands on an authorized OpenShift 4.20 lab using an identity with the required permissions. Never use destructive commands against production without change approval.

```
oc create deployment hello --image=registry.access.redhat.com/ubi9/httpd-24
oc set resources deployment/hello --requests=cpu=100m,memory=128Mi
--limits=cpu=500m,memory=512Mi
oc rollout status deployment/hello
oc rollout history deployment/hello
```

Expected verification: Pod Ready, ReplicaSet desired/current match and deployment Available.

Failure investigation: CrashLoopBackOff: inspect last termination reason, previous logs, probes, env, secret references and OOMKilled.

Interview Q: Why do readiness and liveness differ?

Answer: Readiness controls traffic eligibility; liveness determines whether kubelet restarts a container.

Scenario drill — senior engineer reasoning

Situation: This component becomes unhealthy immediately after a scheduled infrastructure change. First determine blast radius, inspect timestamps and last known-good state. Gather read-only evidence, check upstream dependencies, and propose the lowest-risk corrective action. State which signals would confirm the hypothesis and how you would validate recovery. Avoid guessing a root cause.

Documentation checkpoint: verify version-specific APIs, flags, supported recovery actions and release caveats in the official Red Hat 4.20 manuals before production execution.

12. Services, Routes and DNS

Concept and internal working | Principle

ClusterIP Service is a stable virtual destination selecting ready endpoints via labels. Route exposes HTTP(S) through ingress controller; edge, passthrough and re-encrypt TLS have different trust boundaries. Traffic: client -> DNS -> LB -> router -> Service -> pod.

Production design and safe operating workflow

Step 1 — Identify desired state and dependencies. Step 2 — Verify authentication, release, permissions and prerequisites. Step 3 — Apply only approved configuration in a test environment. Step 4 — Inspect controller reconciliation and component logs. Step 5 — Validate functionality and document rollback or recovery plan. Hinglish: sirf command chalana enough nahi; command ke peeche control loop aur failure domain samajhna zaroori hai.

Hands-on lab: Services, Routes and DNS

Objective and prerequisites: Run these commands on an authorized OpenShift 4.20 lab using an identity with the required permissions. Never use destructive commands against production without change approval.

```
oc expose deployment hello --port=8080
oc expose service hello
oc get svc,route,endpointslices
oc describe route hello
oc get ingresscontroller -n openshift-ingress-operator
```

Expected verification: Route hostname resolves to ingress LB and backend endpoints are Ready.

Failure investigation: 503 route: inspect router logs, route admitted condition, Service selector, EndpointSlices, targetPort and application readiness.

Interview Q: Why can Service exist but no traffic pass?

Answer: A Service is only a stable abstraction; wrong selector, no ready endpoints, policy or wrong targetPort breaks traffic.

Scenario drill — senior engineer reasoning

Situation: This component becomes unhealthy immediately after a scheduled infrastructure change. First determine blast radius, inspect timestamps and last known-good state. Gather read-only evidence, check upstream dependencies, and propose the lowest-risk corrective action. State which signals would confirm the hypothesis and how you would validate recovery. Avoid guessing a root cause.

Documentation checkpoint: verify version-specific APIs, flags, supported recovery actions and release caveats in the official Red Hat 4.20 manuals before production execution.

13. OVN-Kubernetes networking

Concept and internal working | Principle

OVN uses logical switches, routers and distributed policy with node encapsulation; same-node and cross-node pod traffic follow different data paths. DNS, NetworkPolicy, EgressIP, MTU, Geneve and external routes all matter. Default-deny policy requires explicit allows for required flows.

Production design and safe operating workflow

Step 1 — Identify desired state and dependencies. Step 2 — Verify authentication, release, permissions and prerequisites. Step 3 — Apply only approved configuration in a test environment. Step 4 — Inspect controller reconciliation and component logs. Step 5 — Validate functionality and document rollback or recovery plan. Hinglish: sirf command chalana enough nahi; command ke peeche control loop aur failure domain samajhna zaroori hai.

Hands-on lab: OVN-Kubernetes networking

Objective and prerequisites: Run these commands on an authorized OpenShift 4.20 lab using an identity with the required permissions. Never use destructive commands against production without change approval.

```
oc get network.operator cluster -o yaml
oc get network.config cluster -o yaml
oc get networkpolicy -A
oc get pods -n openshift-ovn-kubernetes
oc get svc -n openshift-dns
```

Expected verification: CNI pods healthy, pod DNS and cross-node connectivity validated from debug pods.

Failure investigation: Timeout: distinguish DNS failure, Service selector, policy drop, overlay encapsulation, SG/NACL and MTU blackhole.

Interview Q: Why can pods communicate on same node but not across nodes?

Answer: Overlay/underlay reachability, encapsulation ports, node routes and MTU can fail only on cross-node paths.

Scenario drill — senior engineer reasoning

Situation: This component becomes unhealthy immediately after a scheduled infrastructure change. First determine blast radius, inspect timestamps and last known-good state. Gather read-only evidence, check upstream dependencies, and propose the lowest-risk corrective action. State which signals would confirm the hypothesis and how you would validate recovery. Avoid guessing a root cause.

Documentation checkpoint: verify version-specific APIs, flags, supported recovery actions and release caveats in the official Red Hat 4.20 manuals before production execution.

14. Secrets, RBAC and SCC

Concept and internal working | Principle

Secrets are API objects; base64 is encoding, not encryption. Use external secret management where appropriate, etcd encryption, audit, scoped RBAC and restricted SCC. OpenShift SCC governs privilege, UID, SELinux, capabilities and host access.

Production design and safe operating workflow

Step 1 — Identify desired state and dependencies. Step 2 — Verify authentication, release, permissions and prerequisites. Step 3 — Apply only approved configuration in a test environment. Step 4 — Inspect controller reconciliation and component logs. Step 5 — Validate functionality and document rollback or recovery plan. Hinglish: sirf command chalana enough nahi; command ke peeche control loop aur failure domain samajhna zaroori hai.

Hands-on lab: Secrets, RBAC and SCC

Objective and prerequisites: Run these commands on an authorized OpenShift 4.20 lab using an identity with the required permissions. Never use destructive commands against production without change approval.

```
oc create secret generic app-creds --from-literal=username=demo
--from-literal=password=REPLACE -n payments
oc auth can-i get secrets --as=system:serviceaccount:payments:default -n payments
oc get scc
oc adm policy who-can use scc/privileged
```

Expected verification: Only intended service accounts access Secrets and elevated SCC; rotate demo credentials after lab.

Failure investigation: Permission denied: inspect subject, rolebinding, verbs/resources, service account and SCC admission events.

Interview Q: Why is cluster-admin dangerous for CI?

Answer: It allows broad cluster control; use dedicated service accounts and narrowly scoped Role/RoleBinding.

Scenario drill — senior engineer reasoning

Situation: This component becomes unhealthy immediately after a scheduled infrastructure change. First determine blast radius, inspect timestamps and last known-good state. Gather read-only evidence, check upstream dependencies, and propose the lowest-risk corrective action. State which signals would confirm the hypothesis and how you would validate recovery. Avoid guessing a root cause.

Documentation checkpoint: verify version-specific APIs, flags, supported recovery actions and release caveats in the official Red Hat 4.20 manuals before production execution.

15. Storage and CSI

Concept and internal working | Principle

PVC requests capacity/access modes; StorageClass provisions volumes through CSI. AWS EBS is AZ-scoped and generally ReadWriteOnce; pod scheduling must respect volume topology. EFS supports different sharing characteristics. Backup of etcd does not back up PV application data.

Production design and safe operating workflow

Step 1 — Identify desired state and dependencies. Step 2 — Verify authentication, release, permissions and prerequisites. Step 3 — Apply only approved configuration in a test environment. Step 4 — Inspect controller reconciliation and component logs. Step 5 — Validate functionality and document rollback or recovery plan. Hinglish: sirf command chalana enough nahi; command ke peeche control loop aur failure domain samajhna zaroori hai.

Hands-on lab: Storage and CSI

Objective and prerequisites: Run these commands on an authorized OpenShift 4.20 lab using an identity with the required permissions. Never use destructive commands against production without change approval.

```
oc get sc
oc get pvc,pv -A
oc get pods -n openshift-cluster-csi-drivers
oc describe pvc <claim> -n payments
oc get volumesnapshotclasses
```

Expected verification: PVC Bound, CSI controller/node pods healthy and topology compatible.

Failure investigation: Multi-Attach: inspect pod placement, VolumeAttachment, AZ, RWO semantics and stale attachment; avoid unsafe detach.

Interview Q: Why can a restored PVC fail in another AZ?

Answer: EBS volume topology is tied to its availability zone; restore/provision must align with node placement.

Scenario drill — senior engineer reasoning

Situation: This component becomes unhealthy immediately after a scheduled infrastructure change. First determine blast radius, inspect timestamps and last known-good state. Gather read-only evidence, check upstream dependencies, and propose the lowest-risk corrective action. State which signals would confirm the hypothesis and how you would validate recovery. Avoid guessing a root cause.

Documentation checkpoint: verify version-specific APIs, flags, supported recovery actions and release caveats in the official Red Hat 4.20 manuals before production execution.

16. Operators and OLM

Concept and internal working | Principle

Operators package domain-specific controllers and CRDs. OLM manages catalog subscriptions, InstallPlans and CSVs; installed Operator compatibility matters before cluster updates. Cluster Operators differ from optional OLM-managed Operators.

Production design and safe operating workflow

Step 1 — Identify desired state and dependencies. Step 2 — Verify authentication, release, permissions and prerequisites. Step 3 — Apply only approved configuration in a test environment. Step 4 — Inspect controller reconciliation and component logs. Step 5 — Validate functionality and document rollback or recovery plan. Hinglish: sirf command chalana enough nahi; command ke peeche control loop aur failure domain samajhna zaroori hai.

Hands-on lab: Operators and OLM

Objective and prerequisites: Run these commands on an authorized OpenShift 4.20 lab using an identity with the required permissions. Never use destructive commands against production without change approval.

```
oc get csv,subscriptions,installplans -A
oc get co
oc describe subscription <name> -n <namespace>
```

Expected verification: CSV Succeeded, expected CRDs installed, reconciliation healthy.

Failure investigation: InstallPlan stalled: inspect catalog sources, image pulls, approval strategy, RBAC and resolution errors.

Interview Q: How does Operator differ from a Helm chart?

Answer: Operator continuously reconciles custom resource lifecycle; Helm primarily templates and applies release resources.

Scenario drill — senior engineer reasoning

Situation: This component becomes unhealthy immediately after a scheduled infrastructure change. First determine blast radius, inspect timestamps and last known-good state. Gather read-only evidence, check upstream dependencies, and propose the lowest-risk corrective action. State which signals would confirm the hypothesis and how you would validate recovery. Avoid guessing a root cause.

Documentation checkpoint: verify version-specific APIs, flags, supported recovery actions and release caveats in the official Red Hat 4.20 manuals before production execution.

17. Updates and CVO

Concept and internal working | Principle

CVO reconciles release payload; MCO may drain/reboot nodes. Review Upgradeable condition, operator compatibility, MCP health, PDBs, workload availability, etcd backup and supported update graph. Cluster version rollback is not supported as a routine recovery method.

Production design and safe operating workflow

Step 1 — Identify desired state and dependencies. Step 2 — Verify authentication, release, permissions and prerequisites. Step 3 — Apply only approved configuration in a test environment. Step 4 — Inspect controller reconciliation and component logs. Step 5 — Validate functionality and document rollback or recovery plan. Hinglish: sirf command chalana enough nahi; command ke peeche control loop aur failure domain samajhna zaroori hai.

Hands-on lab: Updates and CVO

Objective and prerequisites: Run these commands on an authorized OpenShift 4.20 lab using an identity with the required permissions. Never use destructive commands against production without change approval.

```
oc adm upgrade
oc get clusterversion -o yaml
oc get co
oc get mcp
oc get pdb -A
```

Expected verification: All operators healthy and desired version available through supported channel.

Failure investigation: Stuck upgrade: identify blocking operator, MCP node, drain, certificate or payload error; follow supported recovery and engage Red Hat support.

Interview Q: Why not downgrade after a failed update?

Answer: Schema and component changes may not be reversible; supported remediation continues or repairs update rather than arbitrary downgrade.

Scenario drill — senior engineer reasoning

Situation: This component becomes unhealthy immediately after a scheduled infrastructure change. First determine blast radius, inspect timestamps and last known-good state. Gather read-only evidence, check upstream dependencies, and propose the lowest-risk corrective action. State which signals would confirm the hypothesis and how you would validate recovery. Avoid guessing a root cause.

Documentation checkpoint: verify version-specific APIs, flags, supported recovery actions and release caveats in the official Red Hat 4.20 manuals before production execution.

18. Monitoring and logging

Concept and internal working | Principle

OpenShift monitoring uses Prometheus-based metrics and alerting; log collection may use supported logging stack. Golden signals: latency, traffic, errors, saturation. Alert hygiene requires runbooks, severity, ownership and silencing discipline.

Production design and safe operating workflow

Step 1 — Identify desired state and dependencies. Step 2 — Verify authentication, release, permissions and prerequisites. Step 3 — Apply only approved configuration in a test environment. Step 4 — Inspect controller reconciliation and component logs. Step 5 — Validate functionality and document rollback or recovery plan. Hinglish: sirf command chalana enough nahi; command ke peeche control loop aur failure domain samajhna zaroori hai.

Hands-on lab: Monitoring and logging

Objective and prerequisites: Run these commands on an authorized OpenShift 4.20 lab using an identity with the required permissions. Never use destructive commands against production without change approval.

```
oc get pods -n openshift-monitoring
oc get prometheusrules -A
oc get alertmanager -n openshift-monitoring
oc adm top pods -A
oc adm must-gather --dest-dir=./must-gather
```

Expected verification: Monitoring pods healthy, alerts explain real symptoms and must-gather captured when escalated.

Failure investigation: Metrics missing: inspect scrape targets, service monitors, RBAC, storage, network policies and retention.

Interview Q: What distinguishes symptom alert from cause alert?

Answer: Symptom indicates user impact; cause alerts help explain underlying failure but may be noisy or misleading alone.

Scenario drill — senior engineer reasoning

Situation: This component becomes unhealthy immediately after a scheduled infrastructure change. First determine blast radius, inspect timestamps and last known-good state. Gather read-only evidence, check upstream dependencies, and propose the lowest-risk corrective action. State which signals would confirm the hypothesis and how you would validate recovery. Avoid guessing a root cause.

Documentation checkpoint: verify version-specific APIs, flags, supported recovery actions and release caveats in the official Red Hat 4.20 manuals before production execution.

19. etcd and disaster recovery

Concept and internal working | Principle

etcd stores cluster API state with Raft consensus. 3 members need quorum 2; 2 failed members lose write availability. Snapshot and static pod resources are both required for documented restoration. etcd restore is destructive and last resort, not first response to an API problem.

Production design and safe operating workflow

Step 1 — Identify desired state and dependencies. Step 2 — Verify authentication, release, permissions and prerequisites. Step 3 — Apply only approved configuration in a test environment. Step 4 — Inspect controller reconciliation and component logs. Step 5 — Validate functionality and document rollback or recovery plan. Hinglish: sirf command chalana enough nahi; command ke peeche control loop aur failure domain samajhna zaroori hai.

Hands-on lab: etcd and disaster recovery

Objective and prerequisites: Run these commands on an authorized OpenShift 4.20 lab using an identity with the required permissions. Never use destructive commands against production without change approval.

```
oc get pods -n openshift-etcd
oc get co etcd
oc debug node/<master> -- chroot /host /usr/local/bin/cluster-backup.sh /home/core/backup
# VERIFY backup files and copy to encrypted off-node storage
```

Expected verification: Backup includes snapshot_*.db and static_kuberesources_*.tar.gz; validate retention and restore drill.

Failure investigation: If quorum lost, diagnose node failure and connectivity first; follow exact Red Hat recovery flow, same z-stream backup and split-brain prevention.

Interview Q: Why is etcd snapshot not an application backup?

Answer: It preserves API object state, not the full data stored on persistent volumes or external databases.

Scenario drill — senior engineer reasoning

Situation: This component becomes unhealthy immediately after a scheduled infrastructure change. First determine blast radius, inspect timestamps and last known-good state. Gather read-only evidence, check upstream dependencies, and propose the lowest-risk corrective action. State which signals would confirm the hypothesis and how you would validate recovery. Avoid guessing a root cause.

Documentation checkpoint: verify version-specific APIs, flags, supported recovery actions and release caveats in the official Red Hat 4.20 manuals before production execution.

20. Jenkins and GitOps

Concept and internal working | Principle

Jenkins builds/tests/scans and publishes immutable images; OpenShift deploys through scoped service account. Avoid injecting kubeadmin credentials into Jenkins. GitOps via Argo CD reconciles desired manifests; separate build pipeline from deploy approval and drift policy.

Production design and safe operating workflow

Step 1 — Identify desired state and dependencies. Step 2 — Verify authentication, release, permissions and prerequisites. Step 3 — Apply only approved configuration in a test environment. Step 4 — Inspect controller reconciliation and component logs. Step 5 — Validate functionality and document rollback or recovery plan. Hinglish: sirf command chalana enough nahi; command ke peeche control loop aur failure domain samajhna zaroori hai.

Hands-on lab: Jenkins and GitOps

Objective and prerequisites: Run these commands on an authorized OpenShift 4.20 lab using an identity with the required permissions. Never use destructive commands against production without change approval.

```
oc -n payments create sa jenkins-deployer
oc -n payments create rolebinding jenkins-edit --clusterrole=edit
--serviceaccount=payments:jenkins-deployer
oc -n payments auth can-i patch deployments
--as=system:serviceaccount:payments:jenkins-deployer
oc -n payments rollout status deployment/hello
```

Expected verification: CI account can deploy only within intended namespace; audit image digest and rollout outcome.

Failure investigation: Pipeline denied: inspect token audience/expiry, namespace, RoleBinding, API URL, TLS trust and Jenkins credential scope.

Interview Q: Why separate CI from CD?

Answer: Build provenance, approvals, least privilege and controlled reconciliation reduce risk of arbitrary code changes reaching production.

Scenario drill — senior engineer reasoning

Situation: This component becomes unhealthy immediately after a scheduled infrastructure change. First determine blast radius, inspect timestamps and last known-good state. Gather read-only evidence, check upstream dependencies, and propose the lowest-risk corrective action. State which signals would confirm the hypothesis and how you would validate recovery. Avoid guessing a root cause.

Documentation checkpoint: verify version-specific APIs, flags, supported recovery actions and release caveats in the official Red Hat 4.20 manuals before production execution.

21. Production troubleshooting

Concept and internal working | Principle

Senior RCA starts with impact and blast radius, time of onset, recent changes and correlated evidence. Compare API, node, CNI, storage and application signals; record timestamps and avoid unapproved destructive remediation.

Production design and safe operating workflow

Step 1 — Identify desired state and dependencies. Step 2 — Verify authentication, release, permissions and prerequisites. Step 3 — Apply only approved configuration in a test environment. Step 4 — Inspect controller reconciliation and component logs. Step 5 — Validate functionality and document rollback or recovery plan. Hinglish: sirf command chalana enough nahi; command ke peeche control loop aur failure domain samajhna zaroori hai.

Hands-on lab: Production troubleshooting

Objective and prerequisites: Run these commands on an authorized OpenShift 4.20 lab using an identity with the required permissions. Never use destructive commands against production without change approval.

```
oc get co
oc get nodes
oc get events -A --sort-by=.lastTimestamp
oc get pods -A --field-selector=status.phase!=Running
oc adm must-gather --dest-dir=./case-001
```

Expected verification: Collect timeline, error messages, affected namespaces and recovery validation.

Failure investigation: Do not infer root cause from a single log line; compare dependency graph and reproduce in lab.

Interview Q: How do you manage a P1 outage?

Answer: Establish incident command, preserve evidence, mitigate impact, communicate, validate service restoration and publish corrective actions.

Scenario drill — senior engineer reasoning

Situation: This component becomes unhealthy immediately after a scheduled infrastructure change. First determine blast radius, inspect timestamps and last known-good state. Gather read-only evidence, check upstream dependencies, and propose the lowest-risk corrective action. State which signals would confirm the hypothesis and how you would validate recovery. Avoid guessing a root cause.

Documentation checkpoint: verify version-specific APIs, flags, supported recovery actions and release caveats in the official Red Hat 4.20 manuals before production execution.

22. Senior interview simulation

Concept and internal working | Principle

Interviewers test mental models and risk awareness, not just command memorization. Explain what you observe, what hypothesis it supports, what would falsify it and the safe next step.

Production design and safe operating workflow

Step 1 — Identify desired state and dependencies. Step 2 — Verify authentication, release, permissions and prerequisites. Step 3 — Apply only approved configuration in a test environment. Step 4 — Inspect controller reconciliation and component logs. Step 5 — Validate functionality and document rollback or recovery plan. Hinglish: sirf command chalana enough nahi; command ke peeche control loop aur failure domain samajhna zaroori hai.

Hands-on lab: Senior interview simulation

Objective and prerequisites: Run these commands on an authorized OpenShift 4.20 lab using an identity with the required permissions. Never use destructive commands against production without change approval.

```
oc get co; oc get nodes; oc get mcp; oc get csr; oc get pvc -A
# Practice explaining the result of each command without changing state
```

Expected verification: Candidate can trace a failure across AWS LB -> API -> etcd -> controller -> node -> workload.

Failure investigation: Practice scenario questions without a working cluster by labeling outputs as hypothetical.

Interview Q: How do you prioritize a degraded operator?

Answer: Assess customer impact and dependency criticality, inspect conditions/events/logs, compare recent changes, choose reversible remediation and verify.

Scenario drill — senior engineer reasoning

Situation: This component becomes unhealthy immediately after a scheduled infrastructure change. First determine blast radius, inspect timestamps and last known-good state. Gather read-only evidence, check upstream dependencies, and propose the lowest-risk corrective action. State which signals would confirm the hypothesis and how you would validate recovery. Avoid guessing a root cause.

Documentation checkpoint: verify version-specific APIs, flags, supported recovery actions and release caveats in the official Red Hat 4.20 manuals before production execution.

Appendix A — Annotated production YAML

Deployment with probes, security context and resources

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: web
  namespace: payments
spec:
  replicas: 3
  selector:
    matchLabels: {app: web}
  template:
    metadata:
      labels: {app: web}
    spec:
      containers:
      - name: web
        image: registry.access.redhat.com/ubi9/httpd-24:latest
        ports:
        - containerPort: 8080
        resources:
          requests: {cpu: 100m, memory: 128Mi}
          limits: {cpu: 500m, memory: 512Mi}
        readinessProbe:
          httpGet: {path: /, port: 8080}
          initialDelaySeconds: 5
        securityContext:
          allowPrivilegeEscalation: false
          capabilities: {drop: [ALL]}
```

replicas=3 supports rolling maintenance if capacity and PDB permit. selector must match pod labels. Requests affect scheduling, limits affect cgroups. Readiness controls traffic membership. Pin images by digest in production; :latest here is a lab convenience only.

Service and Route

```
apiVersion: v1
kind: Service
metadata: {name: web, namespace: payments}
spec:
  selector: {app: web}
  ports:
  - {name: http, port: 80, targetPort: 8080}
---
apiVersion: route.openshift.io/v1
kind: Route
metadata: {name: web, namespace: payments}
spec:
  to: {kind: Service, name: web}
  port: {targetPort: http}
  tls: {termination: edge, insecureEdgeTerminationPolicy: Redirect}
```

Service port is client-facing inside cluster; targetPort reaches container. Route selects service and TLS edge terminates at router. For end-to-end encryption choose re-encrypt with correct destination CA, or passthrough with TLS on pod.

Default deny ingress NetworkPolicy

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata: {name: default-deny-ingress, namespace: payments}
spec:
  podSelector: {}
  policyTypes: [Ingress]
```

This denies inbound traffic to selected namespace pods unless other policies allow it. Test DNS, router-to-pod and monitoring flows before enforcing restrictive policies.

Secret from command and secure mounting

```
oc -n payments create secret generic db-auth \
  --from-literal=username=app \
  --from-literal=password=REPLACE
oc -n payments set env deployment/web --from=secret/db-auth
oc -n payments rollout status deployment/web
```

Avoid literal passwords in shell history in real environments: use approved secrets manager or protected files/STDIN workflow. Kubernetes Secret values are not encrypted simply because they are base64 encoded.

Appendix B — AWS UPI execution checklist and stop points

1. Freeze OCP 4.20 z-stream, architecture, IAM mode, region, AZs and RHCOS AMI
2. Review official CloudFormation templates and actual parameter keys
3. Create VPC, public/private subnets, route tables, NAT and DNS prerequisites
4. Create load balancers, target groups, DNS and internal MCS exposure
5. Create IAM/security stacks with approved privileges and least access
6. Generate install-config, manifests, ignition and preserve metadata
7. Securely deliver bootstrap ignition and launch bootstrap
8. Launch 3 control plane nodes; monitor bootstrap handoff
9. Remove bootstrap target registrations and temporary stack after success
10. Launch workers, inspect and approve only legitimate CSRs
11. Configure ingress wildcard, registry storage, operators and user access
12. Validate node/AZ failure tolerance, backups, monitoring and restore drill
13. Archive IaC state, secure secrets and document UPI deletion order

Execution note: This appendix is a controlled checklist, NOT a fully parameterized one-click deployment. Official CloudFormation template bodies, AMI, zone IDs and account-specific parameters are required. This book deliberately does not invent them.

Appendix C — Glossary

API server: Validates and persists Kubernetes API objects

CVO: Cluster Version Operator; reconciles release payload

MCO: Machine Config Operator; manages node OS configuration

MCP: MachineConfigPool; node configuration rollout grouping

MCD: Machine Config Daemon; applies node configuration

RHCOS: Red Hat Enterprise Linux CoreOS

Ignition: First-boot node configuration engine

SCC: Security Context Constraints; OpenShift admission security

RBAC: Role Based Access Control

OLM: Operator Lifecycle Manager

CSV: ClusterServiceVersion

OVN: Open Virtual Network; virtualized logical networking

Geneve: Overlay encapsulation protocol used by OVN-Kubernetes

EgressIP: Managed stable egress source IP for selected workloads

PVC: PersistentVolumeClaim

CSI: Container Storage Interface

PDB: PodDisruptionBudget

CSR: CertificateSigningRequest

UPI: User-Provisioned Infrastructure

IPI: Installer-Provisioned Infrastructure

RPO: Recovery Point Objective

RTO: Recovery Time Objective

SLO: Service Level Objective

STS: AWS Security Token Service

Route: OpenShift HTTP(S) ingress resource

etcd: Consensus key-value store for API state

Raft: Consensus algorithm used by etcd

Official documentation and scope limitations

https://docs.redhat.com/en/documentation/openshift_container_platform/4.20/html/installing_on_aws/user-provisioned-infrastructure

https://docs.redhat.com/en/documentation/openshift_container_platform/4.20/html/updating_clusters/performing-a-cluster-update

https://docs.redhat.com/en/documentation/openshift_container_platform/4.20/html-single/etcd/etcd

https://docs.redhat.com/en/documentation/openshift_container_platform/4.20/html/support/troubleshooting

Scope statement: This is a substantial senior-administrator study handbook with real examples and operational reasoning, not an exhaustive replacement for all Red Hat manuals, a tested deployment, or a claim of production certification. Illustrative commands need environment-specific validation.

GLOSSARY

API server
Control-plane endpoint that authenticates, validates and processes Kubernetes API requests.

Bootstrap
Temporary initialization phase and machine used to bring up a new control plane.

CNI
Container Network Interface: plugin specification used for pod networking.

ClusterIP
Virtual Service address for accessing a set of backing pods.

CVO
Cluster Version Operator: coordinates OpenShift release updates.

CSI
Container Storage Interface: storage integration between Kubernetes and drivers.

CSR
Certificate Signing Request used for node or client identity approval.

etcd
Distributed strongly consistent key-value store containing cluster state.

EgressIP
Stable outbound source IP functionality for selected workloads.

GitOps
Declarative operations where Git holds desired state and a controller reconciles it.

Ignition
RHCOS first-boot configuration mechanism.

IngressController
Operator-managed routing entry point for applications.

Jenkins agent
Execution environment running stages of a Jenkins pipeline.

Kubelet
Node agent responsible for pod lifecycle and reporting node status.

MachineConfig
Declarative host configuration for RHCOS nodes.

GLOSSARY

MCO
Machine Config Operator: applies and manages node OS configuration.

MCP
MachineConfigPool: groups nodes for coordinated configuration rollout.

NAT gateway
AWS service providing outbound network address translation.

NetworkPolicy
Kubernetes policy controlling selected pod ingress and egress.

OADP
OpenShift API for Data Protection: backup and restore of cluster resources and supported volumes.

OLM
Operator Lifecycle Manager for installing and managing Operators.

OVN-Kubernetes
OpenShift default software-defined networking implementation.

PVC
PersistentVolumeClaim requesting storage from a StorageClass or PV.

PDB
PodDisruptionBudget limiting voluntary application disruptions.

RBAC
Role-based access control using Roles, ClusterRoles and bindings.

RHCOS
Red Hat Enterprise Linux CoreOS, the OpenShift node operating system.

Route
OpenShift object exposing a Service through the ingress router.

SCC
Security Context Constraints: OpenShift pod admission security mechanism.

ServiceAccount
Identity assigned to pods for API authentication and authorization.

STS
AWS Security Token Service providing temporary credentials.

GLOSSARY

UPI

User-Provisioned Infrastructure: user-managed infrastructure installation approach.

Webhook

HTTP callback used to trigger automated workflows such as Jenkins builds.

OPENSHIFT 4.20 — SENIOR ADMINISTRATOR EXPANDED OPERATIONS VOLUME

Supplement to the original 99-page combined masterbook. This new material is incorporated into the same PDF. Commands are illustrative and must be adapted to the actual cluster, IAM model and 4.20 patch release. No deployment is represented as tested.

Revision scope: UPI, etcd, updates, MCO, Operators, OVN, DNS, Services, Routes, security, IAM, storage, monitoring, Jenkins, GitOps, DR, disconnected clusters, performance and interview drills.

1. AWS UPI deployment: dependencies and execution

Principle and internal working

UPI separates cluster asset generation from infrastructure ownership. Installer-generated manifests and Ignition do not create your VPC, EC2 machines, target groups or DNS. Control-plane bootstrap must be reachable through API and Machine Config Server endpoints. Plan three independent availability zones and a controlled, auditable AWS credential model.

Production workflow

Preflight AWS quotas, IAM, DNS ownership, region-specific RHCOS AMI and outbound registry reachability. Generate install-config, back it up, generate manifests, review mastersSchedulable, then generate Ignition. Deploy official release-specific VPC, networking, security, bootstrap, masters and workers templates in documented order. Observe bootstrap completion before deleting bootstrap. Approve only verified CSRs; complete ingress and registry configuration before final install-complete.

Commands / configuration and expected observations

```
aws sts get-caller-identity
aws ec2 describe-vpcs --filters Name=tag:Name,Values='*ocp*'
aws cloudformation validate-template --template-body file://vpc.yaml
openshift-install create manifests --dir ./ocp-prod
openshift-install create ignition-configs --dir ./ocp-prod
jq -r .infraID ./ocp-prod/metadata.json
openshift-install wait-for bootstrap-complete --dir ./ocp-prod --log-level=debug
export KUBECONFIG=$PWD/ocp-prod/auth/kubeconfig
oc get nodes -o wide; oc get co; oc get csr
```

Hands-on lab and validation

LAB: Draw a dependency graph of VPC -> networking -> security -> bootstrap/masters -> workers. For each official CloudFormation template, run get-template-summary and validate-template, then create a change set and inspect it before execution. Verify private API DNS and TCP 22623 from inside the VPC. Keep stack outputs in a secured inventory; never invent template parameter keys.

Incident response and recovery

If bootstrap stalls, examine CloudFormation events, target health, security group reachability, S3 bootstrap-object permissions, DNS and RHCOS console logs. Separate a failed EC2 launch from an API failure. Do not delete bootstrap until installer explicitly confirms completion. Preserve install assets and cloud logs for support.

Senior interview Q&A

Q: Why can API DNS resolve while installation fails? A: DNS only identifies a load balancer; unhealthy targets, blocked 6443/22623 or missing bootstrap services still prevent API access.

Q: What is the main UPI operational burden? A: Infrastructure lifecycle, DNS, target registration, IAM and recovery are the administrator responsibility.

2. etcd, Raft consensus and disaster recovery

Principle and internal working

etcd is the strongly consistent Kubernetes state store. Raft elects a leader and commits writes only with majority agreement. A three-member voting cluster requires two healthy voting members for quorum; one survivor alone cannot commit state changes. Latency, disk fsync performance and network partitions directly affect API health.

Production workflow

Inspect etcd operator conditions, member pod health, storage pressure and control-plane connectivity. Define scheduled, encrypted, access-controlled etcd backups and separately protect required static-pod resources. Recovery procedure

depends on failure mode: a degraded member is not equivalent to total control-plane loss. Follow the exact supported OpenShift 4.20 recovery guide.

Commands / configuration and expected observations

```
oc get co etcd kube-apiserver
oc get pods -n openshift-etcd -o wide
oc get nodes -l node-role.kubernetes.io/master
oc adm must-gather
# On a healthy control-plane node, follow the release-specific
# cluster-backup.sh documented procedure; do not improvise restore.
# Validate backup artifacts, permissions and off-node retention.
```

Hands-on lab and validation

LAB: In a nonproduction cluster, simulate one control-plane node outage; observe quorum and API availability. Record recovery times and inspect etcd alerts. Practice backup artifact inventory and restore procedure only in a disposable environment.

Incident response and recovery

If API is down, verify load balancer, node reachability, etcd quorum and certificate validity before considering restoration. Never run snapshot restore simply because a single operator is degraded. Recovery can overwrite cluster state; freeze changes, obtain approvals and record RPO/RTO.

Senior interview Q&A

Q: Why is a two-of-three etcd failure critical? A: Majority is lost, so consistent writes and API-dependent operations cannot proceed.

Q: Can an etcd snapshot replace application backups? A: No. It captures Kubernetes cluster state, not necessarily application data in external databases or persistent volumes.

3. ClusterVersion Operator, upgrades and release governance

Principle and internal working

The ClusterVersion Operator (CVO) manages the release payload and reconciles cluster operators toward the desired version. Machine Config Operator (MCO) handles node OS/config transitions, while workload availability depends on replica placement, PodDisruptionBudgets and operator health. Cluster version downgrade is not a normal rollback mechanism.

Production workflow

Before upgrading, verify recommended update edges, ClusterVersion conditions, operator health, machine pools, alerts, available storage and backup status. Review release notes, API deprecations, workload disruption budgets and maintenance windows. During the update monitor CVO and MCP progress and investigate first failing condition rather than forcing the update.

Commands / configuration and expected observations

```
oc get clusterversion version -o yaml
oc adm upgrade
oc get co
oc get mcp
oc get nodes
oc get pdb -A
oc describe clusterversion version
oc get events -A --sort-by=.lastTimestamp
```

Hands-on lab and validation

LAB: In a supported test cluster, document prechecks and compare operator/MCP status before and after a patch upgrade. Record how long a worker drain/reboot and workload rescheduling take.

Incident response and recovery

For a stalled update, inspect ClusterVersion history, operator conditions, MCO and machine-config-daemon logs, node readiness and PDBs. Do not blindly force upgrades or downgrade. Use release-specific supported recovery and escalate when payload reconciliation cannot complete.

Senior interview Q&A

Q: What does CVO reconcile? A: The desired release image payload and managed cluster components/operators.

Q: Why can a healthy API coexist with an incomplete upgrade? A: The control plane may be serving requests while an operator or machine pool remains Progressing or Degraded.

4. MachineConfig, MCO, MCP and RHCOS lifecycle

Principle and internal working

MachineConfig defines supported node-level configuration; MCO renders a combined configuration per pool and machine-config-daemon applies it. Some changes trigger drains and reboots. MachineConfigPools expose Updating, Updated and Degraded conditions. Manual host edits can create configuration drift.

Production workflow

Create a narrowly scoped MachineConfig, review the target pool, confirm rendered config and staged rollout. Prefer one controlled pool at a time for risk management. Respect workload PDBs, node drains and maintenance policies. Use oc debug node for diagnosis where appropriate.

Commands / configuration and expected observations

```
oc get mcp
oc describe mcp worker
oc get mc
oc get nodes -l node-role.kubernetes.io/worker
oc get pods -n openshift-machine-config-operator
oc logs -n openshift-machine-config-operator daemonset/machine-config-daemon --tail=80
oc debug node/worker-0 -- chroot /host systemctl status kubelet
```

Hands-on lab and validation

LAB: Apply a harmless supported worker-pool configuration in a lab, watch MCP transition, inspect rendered configuration and node reboot behavior. Record rollback using a Git-controlled config change.

Incident response and recovery

For MCP Degraded, determine whether failure is render, drain, daemon application, reboot or node return. Check MCD logs and node annotations. Never directly modify /etc on RHCOS as a substitute for MCO.

Senior interview Q&A

Q: Why can MCP Updating remain true? A: Nodes may still be draining, applying rendered configuration, rebooting or failing to report the target config.

Q: What is rendered MachineConfig? A: The MCO-composed effective configuration derived from MachineConfigs targeting a pool.

5. OVN-Kubernetes: pod networking and packet path

Principle and internal working

OVN-Kubernetes uses OVS/OVN logical networking to connect pods and implement policies. Pod traffic within a node traverses virtual switching; cross-node traffic may be encapsulated using Geneve. ClusterIP Services are virtual destinations with backends selected from EndpointSlices. DNS resolution and actual TCP reachability are separate checks.

Production workflow

Trace requests in layers: pod DNS -> Service IP -> EndpointSlice -> backend pod -> node and overlay forwarding -> application listener. For external access add ingress load balancer -> router -> Service -> backend. Check MTU when large packets fail but small probes succeed. Evaluate NetworkPolicy in both ingress and egress directions.

Commands / configuration and expected observations

```
oc get network.operator cluster -o yaml
oc get pods -n openshift-ovn-kubernetes -o wide
oc get svc,endpointslice -n demo
oc get networkpolicy -A
oc get ingresscontroller -n openshift-ingress-operator
oc debug node/worker-0 -- chroot /host ip route
oc get events -A --sort-by=.lastTimestamp
```

Hands-on lab and validation

LAB: Deploy two nginx/httpd pods on different nodes; test direct pod IP, ClusterIP and Route. Apply deny-all policy then selectively allow traffic. Capture before/after symptoms and prove policy behavior.

Incident response and recovery

If DNS resolves but TCP fails, inspect backend readiness and port, EndpointSlices, policies, OVS/OVN health, node firewalls, routing and MTU. Do not conclude DNS is the cause from an application timeout alone.

Senior interview Q&A

Q: Does a Service create a container? A: No. It provides a stable virtual endpoint for selected ready backends.

Q: What is a typical cross-node failure? A: OVN encapsulation blocked, MTU mismatch, node routing failure or a restrictive NetworkPolicy.

6. Services, Routes, TLS and DNS internals

Principle and internal working

A Service groups pod endpoints and offers stable service discovery. ClusterIP is internal, NodePort exposes a port on nodes and LoadBalancer requests supported external integration. OpenShift Route maps hostnames to Services through an ingress controller. Edge, passthrough and re-encrypt termination differ in where TLS ends and which certificates are needed.

Production workflow

Create Deployment, label pods, expose a ClusterIP Service with targetPort matching container listener, then create a Route. Verify selector-to-pod label matching and EndpointSlice membership before testing external DNS and TLS. Use readiness probes to avoid routing to unready application instances.

Commands / configuration and expected observations

```
oc create deployment web --image=registry.access.redhat.com/ubi9/httpd-24
oc expose deployment web --port=8080
oc expose service web
oc get deployment,svc,route,endpointslice -o wide
oc describe route web
oc get pods --show-labels
oc rollout status deployment/web
```

Hands-on lab and validation

LAB: Break the Service selector intentionally; inspect empty EndpointSlices, then fix labels. Test edge and re-encrypt routes with trusted certificates in a controlled cluster.

Incident response and recovery

HTTP 503 at the router commonly indicates no usable backend endpoints; distinguish from TLS hostname mismatch, 404 host routing, application 500 or security group timeout. Inspect router logs and route admission.

Senior interview Q&A

Q: Why can a Route exist but return 503? A: The router may have no ready backend endpoints or cannot reach them.

Q: Edge vs passthrough? A: Edge terminates TLS at router; passthrough sends encrypted traffic to backend for termination.

7. RBAC, SCC, Secrets, OAuth and certificates

Principle and internal working

RBAC answers who may perform API actions; SCC governs security constraints for pod admission in OpenShift. A service account token authenticates a workload and should be scoped to minimal privileges. Kubernetes Secret is an API object, not automatically an encrypted external vault. Certificate trust and rotation must be planned.

Production workflow

Create dedicated service accounts, Roles/RoleBindings and security policies by workload. Check permissions with oc auth can-i. Avoid broad cluster-admin assignments and unnecessary privileged SCC. Integrate enterprise identity with tested break-glass administration. Use approved secret-management and encryption-at-rest strategy.

Commands / configuration and expected observations

```
oc create sa ci-deployer -n demo
oc auth can-i create deployments --as=system:serviceaccount:demo:ci-deployer -n demo
oc get scc
oc adm policy who-can use scc privileged
oc create secret generic app-credentials --from-literal=username=demo --from-
literal=password=REPLACE
oc get secret app-credentials -o yaml
oc get oauth cluster -o yaml
```

Hands-on lab and validation

LAB: Create a namespace-scoped deployer Role and bind it to a service account; demonstrate denied secret read and permitted deployment update. Mount a Secret into a pod without printing its value. Rotate and verify credentials.

Incident response and recovery

For forbidden errors inspect verb/resource/namespace, bindings and service account identity. For SCC denials inspect pod securityContext and granted SCC. Never solve a permission issue by granting cluster-admin without impact assessment.

Senior interview Q&A

Q: Does base64 in Secret provide encryption? A: No. It is encoding; apply etcd encryption, access controls and external secret practices.

Q: RBAC vs SCC? A: RBAC authorizes API operations; SCC constrains security attributes of admitted pods.

8. CSI, EBS, EFS and persistent storage

Principle and internal working

CSI separates Kubernetes storage orchestration from cloud-provider volume operations. StorageClass defines provisioner and binding policy. AWS EBS is typically single-AZ block storage, so topology and scheduling matter. EFS provides network file storage with different access and performance characteristics.

Production workflow

Inspect StorageClass, PVC, PV, CSI controller/node pods and volume attachments. Prefer WaitForFirstConsumer where appropriate to align provisioning with pod placement. Plan backups independently of PVC existence and test restoration.

Commands / configuration and expected observations

```
oc get sc
oc get pvc,pv -A
oc describe pvc data -n demo
oc get volumeattachments.storage.k8s.io
oc get pods -n openshift-cluster-csi-drivers
oc get events -n demo --sort-by=.lastTimestamp
```

Hands-on lab and validation

LAB: Provision a PVC using an approved EBS StorageClass, mount into a pod, write data, reschedule within the same AZ, and inspect binding behavior. Test a snapshot/restore where supported.

Incident response and recovery

Pending PVC: inspect StorageClass, CSI permissions, capacity and zone. Multi-Attach: inspect access mode, attachment state and pod placement. Do not detach live EBS volumes as a first response.

Senior interview Q&A

Q: Why can EBS volume block cross-AZ rescheduling? A: An EBS volume belongs to an availability zone and cannot simply attach in a different AZ.

Q: What does WaitForFirstConsumer solve? A: It delays binding/provisioning until scheduling topology is known.

9. Operators, OLM and ClusterOperator diagnostics

Principle and internal working

Operators reconcile custom resources into desired operational state. OLM manages installation and upgrades of many add-on operators via Subscriptions, InstallPlans and CSVs; not every built-in ClusterOperator is managed by OLM. Reconciliation loops can fail due to missing dependencies, permissions, API changes or unavailable operands.

Production workflow

Identify whether a failure concerns built-in ClusterOperator, an OLM-managed operator or an application operator. Inspect conditions, operator namespace pods, CRDs, Subscriptions, InstallPlans, CSVs and operand logs.

Commands / configuration and expected observations

```
oc get co
oc describe co ingress
oc get subscriptions,installplans,csv -A
oc get crd | head
oc get pods -n openshift-operators
oc get events -A --sort-by=.lastTimestamp
```

Hands-on lab and validation

LAB: Install a supported test operator via approved catalog, inspect CSV and Subscription lifecycle, deploy a custom resource and watch reconciliation.

Incident response and recovery

For Pending InstallPlan, inspect approval mode, catalog health and dependency resolution. For Degraded ClusterOperator inspect its status messages and operand logs before attempting restarts.

Senior interview Q&A

Q: Does OLM manage all cluster operators? A: No. Several platform ClusterOperators are managed by the OpenShift release/CVO rather than OLM.

Q: What is a reconciliation loop? A: Repeated comparison of observed state to desired state with corrective action.

10. Monitoring, alerts, logging and incident response

Principle and internal working

Prometheus collects metrics; Alertmanager groups/routes alerts; cluster monitoring and user-workload monitoring have separate configuration concerns. Logs provide event-level context and metrics provide time-series trends. An SLO converts reliability goals into measurable objectives; an RCA distinguishes triggering events from root causes.

Production workflow

Start with alert severity, affected users and blast radius. Gather ClusterOperator conditions, node metrics, events and relevant logs; establish a timeline. Correlate AWS CloudWatch, ELB target health and EC2 events with cluster symptoms. Preserve must-gather output securely.

Commands / configuration and expected observations

```
oc get co
oc adm top nodes
oc adm top pods -A
oc get events -A --sort-by=.lastTimestamp
oc adm must-gather
# Illustrative PromQL (evaluate in authorized Prometheus UI):
# sum(rate(container_cpu_usage_seconds_total{container!=""}[5m])) by (namespace)
# kube_node_status_condition{condition="Ready",status="true"}
```

Hands-on lab and validation

LAB: Generate controlled CPU load in a lab, record node/pod metrics and alert behavior. Create a runbook with symptom, signal, threshold, owner, escalation and recovery verification.

Incident response and recovery

For a noisy alert, validate the underlying metric and rule rather than suppressing it permanently. For a genuine incident, prioritize restoration and evidence retention; then conduct a blameless RCA with corrective actions.

Senior interview Q&A

Q: Metrics vs logs? A: Metrics summarize trends and thresholds; logs explain individual operations and errors.

Q: What belongs in a runbook? A: Trigger, impact, diagnosis, commands, mitigation, validation, escalation and rollback.

11. Jenkins to OpenShift: secure production CI/CD

Principle and internal working

Jenkins orchestrates build/test/security/deployment steps; OpenShift controls runtime admission and workload rollout. The pipeline should use short-lived or centrally managed credentials, least-privilege service accounts, immutable image digests, registry scanning and an auditable promotion path. Avoid giving Jenkins cluster-admin.

Production workflow

Use Jenkins credentials binding or a supported identity integration. Build and test, scan image, push to approved registry, deploy by digest, monitor rollout and roll back application configuration when necessary. Separate development and production permissions and require approvals for promotion.

Commands / configuration and expected observations

```
pipeline {
  agent any
  stages {
    stage('Test') { steps { sh 'echo Run unit and integration tests' } }
    stage('Scan') { steps { sh 'echo Run approved image scanner' } }
```

```
    stage('Deploy') { steps {
      withCredentials([string(credentialsId: 'ocp-token', variable: 'TOKEN')]) {
        sh '''
          set +x
          oc login "$OCP_API" --token="$TOKEN" --insecure-skip-tls-verify=false
          oc -n demo set image deployment/web web="$IMAGE_DIGEST"
          oc -n demo rollout status deployment/web --timeout=300s
        '''
      }
    } }
  }
}
```

Hands-on lab and validation

LAB: Configure a dedicated CI service account with permissions only to patch deployment and read rollout status in one namespace. Store token in Jenkins credentials. Simulate failed rollout and document recovery using oc rollout undo where supported by the deployment history.

Incident response and recovery

If deployment fails, differentiate authentication (401), authorization (403), image pull, SCC denial, readiness failure and application crash. Never log the token, disable TLS verification or grant unrestricted privileges to bypass a failure.

Senior interview Q&A

Q: Why deploy by digest? A: Digest identifies immutable image content and avoids mutable-tag ambiguity.

Q: How should Jenkins authenticate? A: Use approved, scoped credentials with secure storage and rotation; never hardcode tokens in Jenkinsfile.

12. GitOps, Argo CD and drift reconciliation

Principle and internal working

GitOps treats version-controlled manifests as declared desired state. Argo CD compares Git state with cluster state, surfaces drift and optionally synchronizes. Automated sync requires guardrails around pruning, privileged resources and production changes.

Production workflow

Create a repository with overlays or Helm values per environment. Restrict Argo CD projects, destinations and cluster credentials. Review diffs, apply sync, inspect application health and define rollback through Git history rather than untracked manual edits.

Commands / configuration and expected observations

```
oc get applications.argoproj.io -A
oc get pods -n openshift-gitops
oc get events -n openshift-gitops --sort-by=.lastTimestamp
# Example GitOps application manifest is cluster/environment specific;
# validate apiVersion and installed operator before applying.
```

Hands-on lab and validation

LAB: Deploy a ConfigMap and Deployment through a test GitOps Application. Change a replica count manually, observe OutOfSync and restore desired state through Git.

Incident response and recovery

When synchronization fails inspect repository access, rendered manifests, project RBAC, admission policies, health checks and namespace permissions. Disable destructive auto-prune unless intentionally designed.

Senior interview Q&A

Q: What is drift? A: Observed cluster state differs from Git-declared desired state.

Q: Why is Git rollback not always sufficient? A: Database schema, persistent data and external side effects may need separate migration/restore plans.

13. Capacity, performance and scheduler decisions

Principle and internal working

Scheduler placement depends on requested resources, taints/tolerations, affinity, topology constraints and storage topology. CPU limits can cause throttling, memory limits cause OOM kills, and quotas prevent resource exhaustion by one tenant. HPA adjusts replica counts but cannot create node capacity by itself.

Production workflow

Baseline application requests/limits using observed utilization, enforce LimitRanges and ResourceQuotas, check overcommit and PDBs, and model AZ failure capacity. Investigate CPU pressure, memory pressure, disk pressure and pending pods separately.

Commands / configuration and expected observations

```
oc adm top nodes
oc adm top pods -A
oc describe node worker-0
oc get quota,limitrange -A
oc get hpa -A
oc describe pod POD -n NAMESPACE
oc get pdb -A
```

Hands-on lab and validation

LAB: Deploy a workload with too-large memory request, inspect FailedScheduling, reduce request and verify placement. Then trigger controlled OOM in a lab and compare OOMKilled with CPU throttling.

Incident response and recovery

If a pod is Pending, start with Events: insufficient CPU/memory, taints, affinity, quota or PVC topology. Do not add nodes before identifying the constraint.

Senior interview Q&A

Q: Request vs limit? A: Request informs scheduling and guaranteed allocation accounting; limit bounds runtime resource use.

Q: Can HPA solve full nodes? A: No. HPA adds pods; node autoscaling/capacity planning must supply schedulable resources.

14. Disconnected installation and restricted-network operations

Principle and internal working

Disconnected OpenShift requires mirrored release payloads and operator catalogs, trusted internal registry certificates, controlled DNS and a planned update process. A cluster with no direct internet still needs reliable access to internal registries, identity, NTP and other approved dependencies.

Production workflow

Prepare mirror registry and storage, mirror exact supported release images, configure image digest mirror policies and trusted CA bundles, validate pull access from nodes, then install or update using documented disconnected workflows. Track catalog/operator compatibility.

Commands / configuration and expected observations

```
oc get imagedigestmirrorsets
oc get imagecontentsourcepolicies
oc get proxy cluster -o yaml
oc get image.config.openshift.io/cluster -o yaml
oc get co image-registry
# Mirror commands depend on supported oc-mirror version and release.
```

Hands-on lab and validation

LAB: Mirror a small approved image set to a private registry; verify a pod can pull with restricted egress and trusted CA. Record what fails when trust is missing.

Incident response and recovery

ImagePullBackOff can indicate registry DNS, TLS trust, pull secret, architecture mismatch, mirror mapping or missing digest. Examine pod events and registry access before modifying cluster-wide image configuration.

Senior interview Q&A

Q: Is disconnected equal to no networking? A: No. Internal DNS, registries, APIs and cluster communication remain necessary.

Q: Why mirror by digest? A: Digest pinning ensures content identity across registries.

15. Operational security and compliance

Principle and internal working

Production security is layered: IAM/STS, VPC segmentation, API authorization, SCC, NetworkPolicy, Secrets management, image provenance, node hardening and audit logging. Controls must be tested for effectiveness, not only documented.

Production workflow

Implement least privilege, separate administrative roles, rotate certificates and credentials, define audit retention, scan images, review cluster-admin grants and privileged workloads, and record evidence for compliance. Restrict break-glass access and log every use.

Commands / configuration and expected observations

```
oc get clusterrolebindings
oc adm policy who-can create pods -n demo
oc adm policy who-can use scc privileged
oc get networkpolicy -A
oc get apiservers.config.openshift.io cluster -o yaml
oc get kubeapiserver.operator.openshift.io cluster -o yaml
```

Hands-on lab and validation

LAB: Audit privileged SCC users, cluster-admin bindings and secrets access in a test namespace. Produce findings with severity, affected objects, remediation and verification.

Incident response and recovery

For suspected credential compromise revoke/rotate the affected credentials, preserve audit evidence, inspect authorization changes and affected workloads, then verify access removal. Coordinate with incident response rather than wiping logs.

Senior interview Q&A

Q: What is least privilege? A: Only the minimum actions and scope needed for the task.

Q: Does NetworkPolicy replace AWS security groups? A: No. They operate at different layers and must be designed together.

16. Application deployment, YAML and lifecycle management

Principle and internal working

A Deployment describes desired replicas and rollout strategy; ReplicaSet maintains pod counts; Pods host containers. Readiness controls service routing, liveness can restart a failed container and startup probe protects slow initialization. ConfigMaps hold nonsensitive configuration; Secrets hold sensitive data with access controls.

Production workflow

Apply versioned manifests, inspect diff, deploy, observe rollout, test Service and Route, then rehearse rollback. Prefer probes, resource requests, graceful shutdown and PDBs. Use immutable image references and avoid manual pod edits.

Commands / configuration and expected observations

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: web
spec:
  replicas: 3
  selector:
    matchLabels: {app: web}
  template:
    metadata:
      labels: {app: web}
    spec:
      containers:
      - name: web
        image: registry.access.redhat.com/ubi9/httpd-24
        ports: [{containerPort: 8080}]
        resources:
          requests: {cpu: 100m, memory: 128Mi}
          limits: {cpu: 500m, memory: 512Mi}
        readinessProbe:
          tcpSocket: {port: 8080}
          initialDelaySeconds: 5
          periodSeconds: 10
```

Hands-on lab and validation

LAB: Apply Deployment; create ClusterIP Service selecting app=web; create Route; verify EndpointSlice, rollout and readiness. Change image to an invalid reference and investigate ImagePullBackOff; restore previous image.

Incident response and recovery

CrashLoopBackOff requires container logs, previous logs, probes, exit codes, config and resources. ImagePullBackOff requires registry reachability, auth, image existence and architecture. Separate rollout failure from router/DNS failure.

Senior interview Q&A

Q: Deployment vs DeploymentConfig? A: Deployment is the standard Kubernetes workload controller; legacy DeploymentConfig has different OpenShift-specific behavior and lifecycle.

Q: Why readiness and liveness differ? A: Readiness controls receiving traffic; liveness triggers restart for unrecoverable container state.

17. Incident simulations: L3 diagnostic decision tree

Principle and internal working

Senior incident handling prioritizes blast radius, user impact, timeline and reversible mitigations. An observable symptom does not establish a root cause. Every hypothesis should be tested against logs, metrics and independent checks.

Production workflow

Use a common incident template: detect -> classify -> scope -> gather evidence -> isolate failing layer -> mitigate -> verify -> RCA -> preventive change. Keep timestamps, commands, change records and customer impact.

Commands / configuration and expected observations

```
# API outage
oc get co; oc get nodes; dig api.ocp-prod.example.com
# 503 Route
oc get route,svc,endpointslice,pods -n demo
# Pod stuck Pending
oc describe pod POD -n demo; oc get quota -n demo
# Node NotReady
oc describe node NODE; oc get mcp
# Storage issue
oc describe pvc PVC -n demo; oc get volumeattachments
```

Hands-on lab and validation

LAB: Run five tabletop drills: API LB target failure, wrong Service selector, Pending PVC due to AZ, MCP Degraded, and bad Jenkins deployment. For each produce a one-page incident report with evidence and rollback.

Incident response and recovery

Avoid broad restarts, arbitrary deletion of etcd resources, approval of all CSRs, or cluster-admin grants. Restore safe service first and then investigate systemic causes.

Senior interview Q&A

Q: How do you distinguish symptom from cause? A: Confirm each layer with independent evidence; for example a 503 may be caused by empty endpoints rather than the router itself.

Q: What does senior-level RCA include? A: Trigger, underlying contributing factors, impact, detection gaps, corrective actions, owners and validation.

18. Senior interview practical assessment and model answers

Principle and internal working

An L3 interview assesses safe decision-making as much as command recall. Explain architecture, state assumptions, request missing evidence, choose reversible diagnostics and identify escalation conditions.

Production workflow

For each answer use the sequence: impact -> checks -> interpretation -> mitigation -> validation -> prevention. Practice explaining why an action is safe and which data could be lost.

Commands / configuration and expected observations

```
# Five-minute interview drill
oc get co
oc get nodes
oc get mcp
oc get events -A --sort-by=.lastTimestamp
oc adm top nodes
oc get pods -A --field-selector=status.phase=Pending
```

Hands-on lab and validation

LAB: Time-box a mock incident to 20 minutes. Require a hypothesis, three supporting signals, mitigation, rollback and post-incident recommendation. Score correctness, safety, clarity and prioritization.

Incident response and recovery

Common weak answers include “restart the pod,” “check logs,” or “increase resources” without identifying which component, what log, what condition or what measurable evidence supports the change.

Senior interview Q&A

Q: API is down and kubectl cannot connect: first step? A: Check DNS, load balancer and control-plane reachability outside the API; inspect node/etcd static pod health through authorized host access.

Q: Two masters fail: what is the risk? A: Three-member etcd loses quorum, preventing normal state writes; use supported quorum recovery, not arbitrary member deletion.

Q: Why would PVC stay Pending? A: Provisioner, credentials, storage topology, capacity, class or binding policy may prevent binding.

Q: Why would MCP stay Updating? A: Drain, rendered config, daemon, reboot, node return or PDB constraints.

Q: How to safely grant Jenkins access? A: Namespace-scoped service account, minimal verbs, secure token management and audited deployment approvals.

Glossary — senior administration terms

API LB — Load balancer serving the Kubernetes API endpoint

CVO — ClusterVersion Operator; release payload reconciliation

MCO — Machine Config Operator; manages node configuration

MCP — MachineConfigPool; groups nodes for config rollout

MCD — Machine Config Daemon running on nodes

RHCOS — Red Hat Enterprise Linux CoreOS node operating system

Ignition — First-boot provisioning configuration system

UPI — User-Provisioned Infrastructure

IPI — Installer-Provisioned Infrastructure

etcd quorum — Majority of voting etcd members required for commits

Raft — Consensus algorithm used by etcd

OVN — Open Virtual Network logical networking system

Geneve — Overlay network encapsulation protocol

EndpointSlice — Kubernetes representation of Service backends

SCC — Security Context Constraints, OpenShift admission control

RBAC — Role-Based Access Control for API authorization

OLM — Operator Lifecycle Manager

CSV — ClusterServiceVersion describing an installed operator

CSI — Container Storage Interface

PDB — PodDisruptionBudget

RPO — Recovery Point Objective; tolerated data loss

RTO — Recovery Time Objective; tolerated outage duration

OADP — OpenShift API for Data Protection

GitOps — Git as versioned desired-state source

SLO — Service Level Objective

STS — AWS Security Token Service short-lived credentials

EgressIP — Stable egress source IP functionality for selected workloads

must-gather — OpenShift diagnostic collection command

Source validation note: For production commands and version-sensitive recovery, verify against Red Hat OpenShift Container Platform 4.20 official documentation and AWS service documentation. This appendix is a training supplement, not a tested installation artifact.